

claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf\_d88615a4-f56\agent-a6029042d6f6bbf07.jsonl`
- SHA-256: `e1771b30f073d053f8cab66138604c84b9f60efb3d13b76977dd7d9d0c2f4605`
- Source modified: `2026-06-07T09:12:48+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf\_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T09:10:30.803Z)

Read these files in C:/Users/avidu/Projects/badminton-highlight-indexer and produce a DENSE, anchor-rich CODE_MAP chunk for the "Tests (test -> subsystem map)" subsystem. Use the format: bullet per file/symbol with anchors written as @path::symbol where @path is the repo-relative path EXACTLY as listed (the current layered layout). Capture: key classes/functions/consts; data contracts (mark with \$); registries / plugin extension points (mark with ?); and footguns/gotchas (mark with ?). Be precise ? only list symbols that actually exist in the files. If a path is a directory (e.g. tests/), enumerate its files and map each test to the subsystem it covers.

Files/paths: tests/

2. assistant (2026-06-07T09:10:32.971Z)

I'll read the tests directory to produce the CODE_MAP chunk.

3. user (2026-06-07T09:10:33.402Z)

tests__pycache____init__.cpython-311.pyc
tests__pycache__\test_base.cpython-311-pytest-9.0.3.pyc
tests__pycache__\test_gemini.cpython-311-pytest-9.0.3.pyc
tests__pycache__\test_gemini_api.cpython-311-pytest-9.0.3.pyc
tests__pycache__\test_video.cpython-311-pytest-9.0.3.pyc
tests__pycache__\test_database.cpython-311-pytest-9.0.3.pyc
tests__pycache__\test_geometry.cpython-311-pytest-9.0.3.pyc
tests__pycache__\test_yolo_hybrid.cpython-311-pytest-9.0.3.pyc
tests__pycache__\test_gemini_api.cpython-313-pytest-9.0.3.pyc
tests__init__.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_metrics.py
tests\test_eval_training.py
tests\test_geometry.py
tests\test_providers.py
tests\test_sport_profiles.py
tests\test_video.py
tests__pycache____init__.cpython-313.pyc
tests__pycache__\test_eval_annotations.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_eval_batch.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_eval_classifier.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_eval_metrics.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_eval_training.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_geometry.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_providers.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_sport_profiles.cpython-313-pytest-9.0.3.pyc

```
tests\__pycache__\test_video.cpython-313-pytest-9.0.3.pyc
tests\test_annotation_providers.py
tests\__pycache__\test_annotation_providers.cpython-313-pytest-9.0.3.pyc
tests\test_rally_slicer.py
tests\__pycache__\test_rally_slicer.cpython-313-pytest-9.0.3.pyc
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\__pycache__\test_calibrate_wasb.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_calibration.cpython-313-pytest-9.0.3.pyc
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_provider_gemini.py
tests\__pycache__\test_export_wasb_segments.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_gemini_refine.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_provider_gemini.cpython-313-pytest-9.0.3.pyc
tests\test_calibrate_local.py
tests\test_distill_local.py
tests\__pycache__\test_calibrate_local.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_distill_local.cpython-313-pytest-9.0.3.pyc
tests\test_automated_provider.py
tests\test_base.py
tests\test_config.py
tests\test_database.py
tests\test_eval_harness.py
tests\test_eval_regression.py
tests\test_gemini.py
tests\test_hit_detector.py
tests\test_rally_gate.py
tests\test_reporting_harvest.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_validators_base.py
tests\test_wasb_cache.py
tests\test_yolo_hybrid.py
tests\__pycache__\test_automated_provider.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_base.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_config.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_database.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_eval_harness.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_eval_regression.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_gemini.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_hit_detector.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_reporting_harvest.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_tracknet_hybrid.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_tracknet_runner.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_validators_base.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_wasb_cache.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_yolo_hybrid.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_reporting_spec_faq.cpython-313-pytest-9.0.3.pyc
tests\test_reporting_spec_faq.py
```

4. assistant (2026-06-07T09:10:36.373Z)

Now let me read all the test files to map them accurately. Let me batch the reads.

5. user (2026-06-07T09:10:37.330Z)

```
1      # Tests package
2
```

6. user (2026-06-07T09:10:37.798Z)

```
1      """Tests for ground-truth rally annotation loading."""
```

```

2
3 import textwrap
4 import pytest
5
6 from backend.eval.annotations import parse_timestamp, load_annotations, write_scaffold
7
8
9 # ----- parse_timestamp
10
11 def test_parse_seconds():
12     assert parse_timestamp("102.5") == pytest.approx(102.5)
13     assert parse_timestamp(" 30 ") == pytest.approx(30.0)
14
15
16 def test_parse_mm_ss():
17     assert parse_timestamp("01:39") == pytest.approx(99.0)
18
19
20 def test_parse_hh_mm_ss():
21     assert parse_timestamp("01:02:03") == pytest.approx(3723.0)
22
23
24 def test_parse_invalid_raises():
25     with pytest.raises(ValueError):
26         parse_timestamp("")
27     with pytest.raises(ValueError):
28         parse_timestamp("abc")
29     with pytest.raises(ValueError):
30         parse_timestamp("1:2:3:4")
31
32
33 # ----- load_annotations
34
35 def _write(tmp_path, text):
36     p = tmp_path / "ann.csv"
37     p.write_text(textwrap.dedent(text))
38     return str(p)
39
40
41 def test_load_with_header_and_mixed_formats(tmp_path):
42     path = _write(tmp_path, """\
43         start,end
44         00:01:12,00:01:39
45         102.5,131.0
46     """)
47     ivs = load_annotations(path)
48     assert ivs == [(72.0, 99.0), (102.5, 131.0)]
49
50
51 def test_load_without_header(tmp_path):
52     path = _write(tmp_path, """\
53         0,10
54         20,30
55     """)
56     assert load_annotations(path) == [(0.0, 10.0), (20.0, 30.0)]
57
58
59 def test_load_skips_blank_and_comment_lines(tmp_path):
60     path = _write(tmp_path, """\
61         start,end
62
63         # this is a comment
64         5,9
65     """)
66     assert load_annotations(path) == [(5.0, 9.0)]

```

```

67
68
69 def test_load_sorts_by_start(tmp_path):
70     path = _write(tmp_path, """\
71         50,60
72         0,10
73     """)
74     assert load_annotations(path) == [(0.0, 10.0), (50.0, 60.0)]
75
76
77 def test_load_rejects_start_after_end(tmp_path):
78     path = _write(tmp_path, "10,5\n")
79     with pytest.raises(ValueError, match="start"):
80         load_annotations(path)
81
82
83 def test_load_rejects_short_row(tmp_path):
84     path = _write(tmp_path, "10\n")
85     with pytest.raises(ValueError, match="expected 'start,end'"):
86         load_annotations(path)
87
88
89 def test_write_scaffold_roundtrips(tmp_path):
90     path = str(tmp_path / "scaffold.csv")
91     write_scaffold(path)
92     # An empty annotation file should load to zero intervals (header only).
93     assert load_annotations(path) == []
94

```

7. user (2026-06-07T09:10:38.274Z)

```

1     import os
2
3     from backend.eval import batch
4
5
6     # ---- path resolution -----
7
8     def test_resolve_relative_windows_path():
9         got = batch.resolve_video_path("./data/videos\\shuttle1.mp4", "/collector")
10        assert got == os.path.normpath("/collector/data/videos/shuttle1.mp4")
11
12
13    def test_resolve_absolute_path_kept():
14        got = batch.resolve_video_path("C:/vids/x.mp4", "/collector")
15        assert got == os.path.normpath("C:/vids/x.mp4")
16
17
18    def test_resolve_none():
19        assert batch.resolve_video_path(None, "/collector") is None
20        assert batch.resolve_video_path("", "/collector") is None
21
22
23    # ---- stage selection -----
24
25    def test_auto_stage_final_only_for_motion():
26        assert batch.default_stages_for("motion", "auto") == ["final"]
27        assert batch.default_stages_for("gemini", "auto") == ["final"]
28
29
30    def test_auto_stage_both_for_two_stage_detector():
31        assert batch.default_stages_for("yolo_hybrid", "auto") == ["candidates", "final"]
32
33
34    def test_explicit_stage_overrides_auto():

```

```

35     assert batch.default_stages_for("motion", "both") == ["candidates", "final"]
36     assert batch.default_stages_for("yolo_hybrid", "final") == ["final"]
37
38
39     # ---- aggregation -----
40
41     def _vid(tp, fp, fn, miou, f1):
42         return {"stages": {"final": {"score": {
43             "true_positives": tp, "false_positives": fp, "false_negatives": fn,
44             "mean_iou": miou, "f1": f1,
45         }}}}}
46
47
48     def test_aggregate_micro_and_macro():
49         per_video = [
50             _vid(tp=8, fp=2, fn=2, miou=0.9, f1=0.8), # P=.8 R=.8 F1=.8
51             _vid(tp=6, fp=0, fn=4, miou=0.8, f1=0.75), # P=1 R=.6
52         ]
53         agg = batch.aggregate(per_video, ["final"])["final"]
54         assert agg["tp"] == 14 and agg["fp"] == 2 and agg["fn"] == 6
55         assert agg["videos"] == 2
56         # micro precision = 14/16, recall = 14/20
57         assert round(agg["micro_precision"], 3) == round(14 / 16, 3)
58         assert round(agg["micro_recall"], 3) == round(14 / 20, 3)
59         # TP-weighted mean IoU = (0.9*8 + 0.8*6)/14
60         assert round(agg["mean_iou"], 4) == round((0.9 * 8 + 0.8 * 6) / 14, 4)
61         # macro F1 = mean of per-video F1
62         assert round(agg["macro_f1"], 4) == round((0.8 + 0.75) / 2, 4)
63
64
65     def test_aggregate_handles_missing_stage_and_zero_division():
66         agg = batch.aggregate([], ["final"])["final"]
67         assert agg["tp"] == 0 and agg["micro_f1"] == 0.0 and agg["mean_iou"] == 0.0
68

```

8. user (2026-06-07T09:10:38.704Z)

```

1     import numpy as np
2     import pytest
3
4     from backend.eval.classifier import LogisticRegression
5
6
7     def _separable_data(n=200, seed=0):
8         """Two roughly-separable clusters: 'rallies' (high feature values) vs not."""
9         rng = np.random.default_rng(seed)
10        pos = rng.normal(loc=[15, 0.4, 0.2], scale=0.5, size=(n // 2, 3))
11        neg = rng.normal(loc=[4, 0.05, 0.02], scale=0.5, size=(n // 2, 3))
12        X = np.vstack([pos, neg])
13        y = np.array([1] * (n // 2) + [0] * (n // 2))
14        return X, y
15
16
17    def test_fit_separates_classes():
18        X, y = _separable_data()
19        clf = LogisticRegression(epochs=800).fit(X, y, feature_names=["dur", "peak",
"mean"])
20        acc = (clf.predict(X) == y).mean()
21        assert acc > 0.95
22
23
24    def test_balanced_weighting_recovers_minority():
25        # 10:1 imbalance ? balanced weighting should still catch the positives.
26        rng = np.random.default_rng(1)
27        pos = rng.normal([15, 0.4], 0.5, size=(20, 2))

```

```

28     neg = rng.normal([4, 0.05], 0.5, size=(200, 2))
29     X = np.vstack([pos, neg]); y = np.array([1] * 20 + [0] * 200)
30     clf = LogisticRegression(epochs=800).fit(X, y, class_weight="balanced")
31     pred = clf.predict(X)
32     recall = pred[:20].mean() # fraction of positives caught
33     assert recall >= 0.8
34
35
36 def test_predict_proba_range():
37     X, y = _separable_data()
38     clf = LogisticRegression(epochs=200).fit(X, y)
39     p = clf.predict_proba(X)
40     assert p.min() >= 0.0 and p.max() <= 1.0
41
42
43 def test_save_load_roundtrip(tmp_path):
44     X, y = _separable_data()
45     clf = LogisticRegression(epochs=300).fit(X, y, feature_names=["a", "b", "c"])
46     path = str(tmp_path / "model.json")
47     clf.save(path)
48     loaded = LogisticRegression.load(path)
49     assert loaded.feature_names == ["a", "b", "c"]
50     np.testing.assert_allclose(loaded.predict_proba(X), clf.predict_proba(X))
51
52
53 def test_predict_before_fit_raises():
54     with pytest.raises(RuntimeError):
55         LogisticRegression().predict_proba([[1, 2, 3]])
56
57
58 def test_fit_empty_raises():
59     with pytest.raises(ValueError):
60         LogisticRegression().fit(np.empty((0, 3)), np.array([]))
61

```

9. user (2026-06-07T09:10:39.698Z)

```

1     """Deterministic unit tests for the pure interval-matching metrics."""
2
3     import pytest
4
5     from backend.eval.metrics import (
6         interval_iou, match_intervals, score, pr_curve,
7     )
8
9
10    # ----- interval_iou
11
12    def test_iou_identical():
13        assert interval_iou((0, 10), (0, 10)) == 1.0
14
15
16    def test_iou_no_overlap():
17        assert interval_iou((0, 5), (5, 10)) == 0.0
18        assert interval_iou((0, 5), (6, 10)) == 0.0
19
20
21    def test_iou_partial():
22        # overlap [5,10]=5, union [0,15]=15 -> 1/3
23        assert interval_iou((0, 10), (5, 15)) == pytest.approx(1 / 3)
24
25
26    def test_iou_contained():
27        # overlap=4, union=10 -> 0.4
28        assert interval_iou((0, 10), (3, 7)) == pytest.approx(0.4)

```

```

29
30
31 def test_iou_degenerate_returns_zero():
32     assert interval_iou((5, 5), (0, 10)) == 0.0
33     assert interval_iou((0, 10), (7, 3)) == 0.0
34
35
36 # ----- match_intervals
37
38 def test_match_one_to_one():
39     preds = [(0, 10), (20, 30)]
40     gts = [(0, 10), (20, 30)]
41     mr = match_intervals(preds, gts, iou_threshold=0.5)
42     assert len(mr.matches) == 2
43     assert mr.unmatched_pred_indices == []
44     assert mr.unmatched_gt_indices == []
45
46
47 def test_match_greedy_prefers_higher_iou():
48     # Two preds overlap one GT; the tighter one should win, the other is a FP.
49     preds = [(0, 9), (0, 10)] # pred1 IoU with gt is higher
50     gts = [(0, 10)]
51     mr = match_intervals(preds, gts, iou_threshold=0.5)
52     assert len(mr.matches) == 1
53     assert mr.matches[0].pred_index == 1 # (0,10) is the exact match
54     assert mr.unmatched_pred_indices == [0]
55
56
57 def test_match_below_threshold_is_unmatched():
58     preds = [(0, 10)]
59     gts = [(8, 20)] # small overlap -> low IoU
60     mr = match_intervals(preds, gts, iou_threshold=0.5)
61     assert mr.matches == []
62     assert mr.unmatched_pred_indices == [0]
63     assert mr.unmatched_gt_indices == [0]
64
65
66 def test_match_one_gt_not_double_claimed():
67     preds = [(0, 10), (0, 10)] # two identical preds, one GT
68     gts = [(0, 10)]
69     mr = match_intervals(preds, gts, iou_threshold=0.5)
70     assert len(mr.matches) == 1
71     assert len(mr.unmatched_pred_indices) == 1 # the duplicate is a false positive
72
73
74 # ----- score
75
76 def test_score_perfect():
77     preds = [(0, 10), (20, 30)]
78     gts = [(0, 10), (20, 30)]
79     s = score(preds, gts, 0.5)
80     assert s.precision == 1.0
81     assert s.recall == 1.0
82     assert s.f1 == 1.0
83     assert s.mean_iou == 1.0
84     assert s.mean_start_error == 0.0
85     assert s.mean_end_error == 0.0
86
87
88 def test_score_with_fp_and_fn():
89     # 1 correct, 1 spurious prediction, 1 missed GT
90     preds = [(0, 10), (100, 110)]
91     gts = [(0, 10), (50, 60)]
92     s = score(preds, gts, 0.5)
93     assert s.true_positives == 1

```

```

94     assert s.false_positives == 1
95     assert s.false_negatives == 1
96     assert s.precision == pytest.approx(0.5)
97     assert s.recall == pytest.approx(0.5)
98     assert s.f1 == pytest.approx(0.5)
99
100
101     def test_score_boundary_error():
102         preds = [(1, 11)] # shifted +1s on both ends
103         gts = [(0, 10)]
104         s = score(preds, gts, 0.5)
105         assert s.true_positives == 1
106         assert s.mean_start_error == pytest.approx(1.0)
107         assert s.mean_end_error == pytest.approx(1.0)
108
109
110     def test_score_empty_predictions():
111         s = score([], [(0, 10)], 0.5)
112         assert s.precision == 0.0
113         assert s.recall == 0.0
114         assert s.f1 == 0.0
115         assert s.false_negatives == 1
116
117
118     def test_score_empty_ground_truth():
119         s = score([(0, 10)], [], 0.5)
120         assert s.precision == 0.0 # tp=0, fp=1
121         assert s.false_positives == 1
122
123
124     def test_score_as_dict_roundtrip():
125         s = score([(0, 10)], [(0, 10)], 0.5)
126         d = s.as_dict()
127         assert d["precision"] == 1.0 and d["iou_threshold"] == 0.5
128
129
130     # ----- pr_curve
131
132     def test_pr_curve_default_thresholds():
133         curve = pr_curve([(0, 10)], [(0, 10)])
134         assert len(curve) == 9
135         assert all(s.f1 == 1.0 for s in curve) # perfect match at every threshold
136
137
138     def test_pr_curve_stricter_threshold_drops_loose_match():
139         preds = [(0, 8)] # IoU with (0,10) = 8/10 = 0.8
140         gts = [(0, 10)]
141         curve = pr_curve(preds, gts, thresholds=[0.5, 0.9])
142         assert curve[0].true_positives == 1 # 0.8 >= 0.5
143         assert curve[1].true_positives == 0 # 0.8 < 0.9
144

```

10. user (2026-06-07T09:10:40.160Z)

```

1     from backend.eval import training
2
3
4     def _row(start, end, peak=0.0, mean=0.0, frames=0, tracks=0):
5         return {"start_time": start, "end_time": end,
6                 "stats": {"peak_velocity": peak, "mean_velocity": mean,
7                           "active_frame_count": frames, "track_id_count": tracks}}
8
9
10    # ---- feature extraction -----
11

```



```

12     def test_extract_yolo_features_shape_and_values():
13         feats = training.extract_yolo_features(_row(10, 25, peak=0.4, mean=0.2, frames=30,
14         tracks=2))
15         assert len(feats) == len(training.YOLO_FEATURE_NAMES)
16         assert feats == [15.0, 0.4, 0.2, 30.0, 2.0]
17
18     def test_extract_yolo_features_missing_stats_defaults_zero():
19         feats = training.extract_yolo_features({"start_time": 0, "end_time": 5})
20         assert feats == [5.0, 0.0, 0.0, 0.0, 0.0]
21
22
23     # ---- labeling -----
24
25     def test_label_candidates_matches_ground_truth():
26         # Two candidates overlap real rallies, one is spurious.
27         cands = [(10.0, 20.0), (100.0, 110.0), (500.0, 505.0)]
28         gt = [(10.0, 20.0), (101.0, 111.0)]
29         labels = training.label_candidates(cands, gt, iou_threshold=0.5)
30         assert labels == [1, 1, 0]
31
32
33     def test_label_candidates_below_threshold_is_negative():
34         # Small overlap -> IoU below 0.5 -> negative.
35         cands = [(10.0, 20.0)]
36         gt = [(18.0, 30.0)]
37         assert training.label_candidates(cands, gt, iou_threshold=0.5) == [0]
38
39
40     def test_label_candidates_one_to_one():
41         # Two candidates over a single GT rally: only the best match is positive.
42         cands = [(10.0, 20.0), (10.5, 20.5)]
43         gt = [(10.0, 20.0)]
44         labels = training.label_candidates(cands, gt, iou_threshold=0.5)
45         assert sum(labels) == 1
46
47
48     # ---- glue -----
49
50     def test_build_training_set():
51         rows = [_row(10, 20, peak=0.5, frames=40), _row(500, 505, peak=0.05, frames=5)]
52         gt = [(10.0, 20.0)]
53         X, y, intervals = training.build_training_set(rows, gt, iou_threshold=0.5)
54         assert y == [1, 0]
55         assert intervals == [(10.0, 20.0), (500.0, 505.0)]
56         assert len(X) == 2 and len(X[0]) == len(training.YOLO_FEATURE_NAMES)
57
58
59     def test_build_training_set_custom_feature_fn():
60         rows = [_row(10, 20)]
61         X, y, _ = training.build_training_set(rows, [(10.0, 20.0)],
62         feature_fn=lambda r: [1.0, 2.0])
63         assert X == [[1.0, 2.0]]
64

```

11. user (2026-06-07T09:10:40.623Z)

```

1     import math
2     from backend.utils.geometry import get_center, get_distance, get_velocity,
3     get_velocity_normalised, calculate_iou
4
5     def test_get_center():
6         bbox = (0.0, 0.0, 10.0, 10.0)
7         assert get_center(bbox) == (5.0, 5.0)
8

```

```

8         bbox2 = (10.0, 20.0, 30.0, 40.0)
9         assert get_center(bbox2) == (20.0, 30.0)
10
11     def test_get_distance():
12         p1 = (0.0, 0.0)
13         p2 = (3.0, 4.0)
14         assert math.isclose(get_distance(p1, p2), 5.0)
15
16     def test_get_velocity():
17         bbox1 = (0.0, 0.0, 10.0, 10.0)
18         bbox2 = (3.0, 4.0, 13.0, 14.0)
19         # Centers: (5,5) and (8,9)
20         # Distance:  $\sqrt{3^2 + 4^2} = 5$ 
21         # FPS: 30
22         # Velocity = 5 * 30 = 150.0
23         assert math.isclose(get_velocity(bbox1, bbox2, 30.0), 150.0)
24
25     def test_get_velocity_normalised():
26         bbox1 = (0.0, 0.0, 10.0, 10.0)
27         bbox2 = (3.0, 4.0, 13.0, 14.0)
28         # raw velocity = 150.0
29         # normalized with width 1000 =  $150.0 / 1000.0 = 0.15$ 
30         assert math.isclose(get_velocity_normalised(bbox1, bbox2, 30.0, 1000.0), 0.15)
31
32         # Check fallback when width <= 0
33         assert math.isclose(get_velocity_normalised(bbox1, bbox2, 30.0, 0.0), 150.0)
34
35     def test_calculate_iou():
36         boxA = (0.0, 0.0, 10.0, 10.0)
37         boxB = (0.0, 0.0, 10.0, 10.0)
38         assert math.isclose(calculate_iou(boxA, boxB), 1.0)
39
40         boxC = (100.0, 100.0, 110.0, 110.0)
41         assert calculate_iou(boxA, boxC) == 0.0
42
43         boxD = (5.0, 5.0, 15.0, 15.0)
44         # intersection: x=[5,10], y=[5,10] => area = 5 * 5 = 25 (Wait, the iou function adds
+1 to width and height)
45         # Let's test actual output based on the specific IoU logic (+1 for bounds)
46         iou = calculate_iou(boxA, boxD)
47         assert iou > 0.0 and iou < 1.0
48

```

12. user (2026-06-07T09:10:41.028Z)

```

1     import pytest
2     from backend.providers import provider_registry
3     from backend.providers.base import AIVideoProvider
4     from backend.providers.gemini import GeminiProvider
5
6     def test_provider_registry():
7         available = provider_registry.list_available()
8         assert "gemini" in available
9
10        # Retrieve provider instance
11        provider = provider_registry.get("gemini", api_key="dummy",
model_name="gemini-2.5-flash")
12        assert isinstance(provider, GeminiProvider)
13

```

13. user (2026-06-07T09:10:44.379Z)

```

1     import pytest
2     from backend.sports import sport_registry
3     from backend.sports.base import SportProfile

```

```

4     from backend.sports.badminton import BadmintonSportProfile
5
6     def test_sport_registry():
7         # Verify badminton is registered
8         available = sport_registry.list_available()
9         assert "badminton" in available
10
11        # Retrieve badminton profile
12        badminton = sport_registry.get("badminton")
13        assert isinstance(badminton, BadmintonSportProfile)
14        assert badminton.name == "badminton"
15
16    def test_badminton_sport_profile():
17        profile = BadmintonSportProfile()
18        prompt = profile.get_prompt(chunk_duration=90.0)
19        assert "badminton" in prompt.lower()
20        assert "decimal seconds" in prompt.lower()
21        assert "90.0" in prompt
22
23        relevance_cfg = profile.get_relevance_config()
24        assert "court_green_lower" in relevance_cfg
25        assert "court_green_upper" in relevance_cfg
26        assert "court_pixels_min_ratio" in relevance_cfg
27

```

14. user (2026-06-07T09:10:45.316Z)

```

1     import pytest
2     from unittest.mock import patch, MagicMock
3     from backend.utils.video import get_video_duration, extract_video_chunk
4
5     @patch("subprocess.check_output")
6     def test_get_video_duration_success(mock_check_output):
7         mock_check_output.return_value = b"123.45\n"
8         dur = get_video_duration("dummy.mp4")
9         assert dur == 123.45
10
11    @patch("subprocess.check_output")
12    def test_get_video_duration_failure(mock_check_output):
13        mock_check_output.side_effect = Exception("ffprobe error")
14        dur = get_video_duration("dummy.mp4")
15        assert dur == 0.0
16
17    @patch("subprocess.run")
18    def test_extract_video_chunk_success(mock_run):
19        success = extract_video_chunk("in.mp4", 10.0, 5.0, "out.mp4")
20        assert success is True
21        mock_run.assert_called_once()
22
23    @patch("subprocess.run")
24    def test_extract_video_chunk_failure(mock_run):
25        mock_run.side_effect = Exception("ffmpeg error")
26        success = extract_video_chunk("in.mp4", 10.0, 5.0, "out.mp4")
27        assert success is False
28

```

15. user (2026-06-07T09:10:45.828Z)

```

1     import pytest
2
3     from backend.annotations import annotation_registry, AnnotationProvider
4     from backend.annotations.base import DONE, FAILED
5     from backend.annotations.file_provider import FileProvider
6
7

```

```

8     def _write_csv(path) -> str:
9         # Mixed decimal + HH:MM:SS forms; header auto-skipped by load_annotations.
10        path.write_text("start,end\n0.0,3.0\n00:00:05,00:00:09\n")
11        return str(path)
12
13
14    # ----- registry -----
15
16    def test_registry_has_file_provider():
17        assert "file" in annotation_registry.list_available()
18        p = annotation_registry.get("file")
19        assert isinstance(p, AnnotationProvider)
20        assert p.name == "file"
21
22
23    def test_registry_unknown_provider_raises():
24        with pytest.raises(KeyError):
25            annotation_registry.get("does-not-exist")
26
27
28    def test_registry_get_passes_kwargs(tmp_path):
29        p = annotation_registry.get("file", csv_dir=str(tmp_path))
30        assert isinstance(p, FileProvider)
31        assert p.csv_dir == str(tmp_path)
32
33
34    # ----- FileProvider: submit / poll / fetch -----
35
36    def test_file_provider_direct_path(tmp_path):
37        csv = _write_csv(tmp_path / "rallies.csv")
38        p = FileProvider()
39        job = p.submit(csv)
40        assert job == csv
41        assert p.poll(job) == DONE
42        assert p.fetch(job) == [(0.0, 3.0), (5.0, 9.0)]
43
44
45    def test_file_provider_dir_resolution(tmp_path):
46        _write_csv(tmp_path / "vid123.csv")
47        p = FileProvider(csv_dir=str(tmp_path))
48        job = p.submit("vid123") # bare video_id, resolved to <dir>/vid123.csv
49        assert job.endswith("vid123.csv")
50        assert p.poll(job) == DONE
51        assert len(p.fetch(job)) == 2
52
53
54    def test_file_provider_missing_file_is_failed(tmp_path):
55        p = FileProvider(csv_dir=str(tmp_path))
56        job = p.submit("nonexistent")
57        assert p.poll(job) == FAILED
58        with pytest.raises(FileNotFoundError):
59            p.fetch(job)
60
61
62    def test_file_provider_annotate_convenience(tmp_path):
63        csv = _write_csv(tmp_path / "rallies.csv")
64        p = FileProvider()
65        assert p.annotate(csv) == [(0.0, 3.0), (5.0, 9.0)]
66
67
68    def test_annotate_raises_when_not_done(tmp_path):
69        # annotate() on a synchronous provider with a missing file must fail loudly,
70        # not silently return [] (which would skew downstream regression scores).
71        p = FileProvider(csv_dir=str(tmp_path))
72        with pytest.raises((RuntimeError, FileNotFoundError)):

```

```

73         p.annotate("nonexistent")
74
75
76     # ----- the contract that makes the whole flow work: provider output == GT shape -----
77
78     def test_provider_output_matches_metrics_interval_shape(tmp_path):
79         """A provider's fetch() output must be directly scorable by the eval harness ?
80         same List[(float, float)] shape metrics.score() expects. This is the keystone
81         that lets one scorer grade product, vendor, and annotator-vs-annotator."""
82         from backend.eval import metrics
83         csv = _write_csv(tmp_path / "gt.csv")
84         gt = FileProvider().annotate(csv)
85         # Feed provider output straight into the scorer as ground truth ? must not raise.
86         s = metrics.score(preds=[(0.0, 3.0), (5.0, 9.0)], gts=gt, iou_threshold=0.5)
87         assert s.true_positives == 2
88         assert s.precision == 1.0 and s.recall == 1.0
89

```

16. user (2026-06-07T09:10:46.270Z)

```

1     """Tests for the rally slicer's pure logic (no ffmpeg)."""
2     import pytest
3
4     from backend.tools.rally_slicer import (
5         load_rally_labels, compute_clip_windows, ClipWindow, BEGIN_PADDING, END_PADDING,
6     )
7
8
9     def _write(tmp_path, rows,
10        header="rally_number,start_time,end_time,start_mmss,end_mmss"):
11         p = tmp_path / "labels.csv"
12         p.write_text(header + "\n" + "\n".join(rows) + "\n", encoding="utf-8")
13         return str(p)
14
15     # ----- load_rally_labels
16
17     def test_load_decimal_and_mmss(tmp_path):
18         csv = _write(tmp_path, [
19             "1,3.0,21.0,,", # decimal
20             "2,,0:29.000,0:32.000", # mm:ss fallback
21         ])
22         labels = load_rally_labels(csv)
23         assert labels["1"] == pytest.approx((3.0, 21.0))
24         assert labels["2"] == pytest.approx((29.0, 32.0))
25
26
27     def test_load_skips_missing_and_degenerate(tmp_path):
28         csv = _write(tmp_path, [
29             "7.5,106.031,,", # MISSED row: no end -> skipped
30             "17,237.0,237.0,,", # zero-duration -> skipped
31             "3,44.5,51.0,,", # good
32         ])
33         labels = load_rally_labels(csv)
34         assert "7.5" not in labels and "17" not in labels
35         assert labels["3"] == pytest.approx((44.5, 51.0))
36
37
38     def test_fractional_ids_preserved(tmp_path):
39         csv = _write(tmp_path, ["16.5,214.0,223.0,,"])
40         assert "16.5" in load_rally_labels(csv)
41
42
43     # ----- compute_clip_windows
44

```

```

45 def test_padding_applied():
46     w = compute_clip_windows({"1": (10.0, 20.0)}, ["1"])
47     assert w == [ClipWindow("1", 10.0 - BEGIN_PADDING, 20.0 + END_PADDING)]
48
49
50 def test_clamp_to_video_bounds():
51     # start near 0 clamps to 0; end past duration clamps to duration
52     w = compute_clip_windows({"1": (0.2, 99.0)}, ["1"], video_duration=99.5)
53     assert w[0].start == 0.0
54     assert w[0].end == 99.5
55
56
57 def test_all_selects_every_rally_in_time_order():
58     labels = {"2": (29.0, 32.0), "1": (3.0, 21.0)}
59     w = compute_clip_windows(labels, ["all"])
60     assert [x.rally for x in w] == ["1", "2"] # ordered by start time, not dict order
61
62
63 def test_subset_and_unknown_skipped():
64     labels = {"1": (3.0, 21.0), "2": (29.0, 32.0), "3": (44.5, 51.0)}
65     w = compute_clip_windows(labels, ["3", "1", "99"]) # 99 unknown -> dropped
66     assert [x.rally for x in w] == ["3", "1"] # preserves requested order
67
68
69 def test_window_entirely_past_end_skipped():
70     w = compute_clip_windows({"1": (200.0, 210.0)}, ["1"], video_duration=100.0)
71     assert w == []
72

```

17. user (2026-06-07T09:10:47.235Z)

```

1     """Tests for the calibration / overfitting-guard eval harness."""
2     import math
3     import pytest
4
5     from backend.eval.calibration import (
6         pct_improvement, kfold_indices, select_best, cross_validate,
7         leave_one_video_out, improvement_report,
8     )
9
10    # Ground truth: 5 rallies. A "config" here is a time offset applied to the GT;
11    # offset 0 reproduces GT exactly (perfect), larger offsets degrade IoU.
12    GTS = [(0.0, 10.0), (20.0, 30.0), (40.0, 50.0), (60.0, 70.0), (80.0, 90.0)]
13
14
15    def predict(offset):
16        return [(s + offset, e + offset) for (s, e) in GTS]
17
18
19    # ----- pct_improvement
20
21    def test_pct_improvement():
22        assert pct_improvement(0.6, 0.75) == pytest.approx(25.0)
23        assert pct_improvement(0.5, 0.5) == 0.0
24        assert math.isinf(pct_improvement(0.0, 0.4)) # from-zero gain
25        assert pct_improvement(0.0, 0.0) == 0.0
26
27
28    # ----- kfold_indices
29
30    def test_kfold_partitions_cover_all_without_overlap():
31        splits = kfold_indices(5, 5)
32        assert len(splits) == 5
33        for train, test in splits:
34            assert test and train

```

```

35         assert set(train).isdisjoint(test)
36         assert set(train) | set(test) == set(range(5))
37
38
39     def test_kfold_caps_k_to_n():
40         assert len(kfold_indices(3, 10)) <= 3
41         assert kfold_indices(0, 5) == []
42
43
44     # ----- select_best
45
46     def test_select_best_picks_perfect_config():
47         cfg, val = select_best(predict, configs=[0.0, 5.0, 20.0], gts=GTS, metric="f1")
48         assert cfg == 0.0 and val == pytest.approx(1.0)
49         # offset 5 = 50% overlap -> IoU 0.33 < 0.5 -> no matches
50         _, bad = select_best(predict, configs=[5.0], gts=GTS, metric="f1")
51         assert bad == 0.0
52
53
54     # ----- cross_validate
55
56     def test_cross_validate_reports_generalization_gap():
57         # default metric=recall (fold-fair); offset 0 detects every held-out rally
58         res = cross_validate(predict, configs=[0.0, 5.0], gts=GTS, k=5)
59         assert res["metric"] == "recall"
60         assert res["folds"] >= 2
61         assert res["mean_test"] == pytest.approx(1.0) # offset 0 generalizes
62         assert res["generalization_gap"] == pytest.approx(0.0, abs=1e-9)
63         assert "picked_configs" in res
64
65
66     def test_cross_validate_too_few():
67         assert "error" in cross_validate(predict, [0.0], gts=[(0.0, 1.0)], k=5)
68
69
70     # ----- leave_one_video_out
71
72     def test_leave_one_video_out():
73         videos = [
74             {"name": "v1", "predict_fn": predict, "gts": GTS},
75             {"name": "v2", "predict_fn": predict, "gts": GTS},
76             {"name": "v3", "predict_fn": predict, "gts": GTS},
77         ]
78         res = leave_one_video_out(videos, configs=[0.0, 5.0], metric="f1")
79         assert res["mean_held_out"] == pytest.approx(1.0)
80         assert len(res["per_video"]) == 3
81         assert all(p["config"] == 0.0 for p in res["per_video"]) # picks the generalizing
82         config
83
84     def test_leave_one_video_out_needs_two():
85         assert "error" in leave_one_video_out([{"name": "v1", "predict_fn": predict, "gts":
86         GTS}], [0.0])
87
88     # ----- improvement_report
89
90     def test_improvement_report():
91         rep = improvement_report(predict, baseline_config=5.0, tuned_config=0.0, gts=GTS)
92         assert rep["baseline"]["f1"] == 0.0
93         assert rep["tuned"]["f1"] == pytest.approx(1.0)
94         assert rep["precision_delta_pts"] == pytest.approx(1.0)
95         assert math.isinf(rep["precision_delta_pct"]) # 0 -> positive
96

```

18. user (2026-06-07T09:10:47.686Z)

```
1      """Tests for the WASB calibration driver (pure parts; no WSL/GPU)."""
2      import csv
3      import pytest
4
5      from backend.eval.calibrate_wasb import (
6          load_golden_gts, make_predict_fn, run, BASELINE,
7      )
8
9
10     def _write_traj(path):
11         """Synthetic trajectory: two moving segments separated by an invisible gap."""
12         rows = [("Frame", "Visibility", "X", "Y")]
13         for i in range(0, 30):          # 0.0-2.9s @10fps: shuttle moving fast
14             rows.append((i, 1, i * 5, 50))
15         for i in range(30, 50):         # dead time: invisible
16             rows.append((i, 0, -1, -1))
17         for i in range(50, 80):         # 5.0-7.9s: moving again
18             rows.append((i, 1, (i - 50) * 5, 50))
19         with open(path, "w", newline="") as f:
20             csv.writer(f).writerows(rows)
21         return str(path)
22
23
24     def _write_golden(path):
25         with open(path, "w", newline="", encoding="utf-8") as f:
26             w = csv.writer(f)
27             w.writerow(["rally_number", "start_time", "end_time", "shots_count"])
28             w.writerow([1, 0.0, 3.0, 8])
29             w.writerow([2, 5.0, 8.0, 6])
30             w.writerow([3, "", "", 5])          # incomplete -> skipped
31         return str(path)
32
33
34     def test_load_golden_gts_skips_incomplete(tmp_path):
35         gts = load_golden_gts(_write_golden(tmp_path / "g.csv"))
36         assert gts == [(0.0, 3.0), (5.0, 8.0)]
37
38
39     def test_predict_fn_finds_moving_segments(tmp_path):
40         pf = make_predict_fn(_write_traj(tmp_path / "t.csv"), fps=10.0, frame_width=100.0)
41         wins = pf(BASELINE)
42         assert len(wins) == 2                # two rallies recovered from the
trajectory
43         assert wins[0][0] < 3.0 and wins[1][0] >= 5.0
44
45
46     def test_run_end_to_end_synthetic(tmp_path):
47         traj = _write_traj(tmp_path / "t.csv")
48         gold = _write_golden(tmp_path / "g.csv")
49         res = run(traj, gold, fps=10.0, frame_width=100.0, iou=0.5)
50         assert res["best_cfg"] is not None
51         assert res["baseline"]["f1"] == pytest.approx(1.0)    # clean synthetic => baseline
already matches
52         assert "cv" in res and "mean_test" in res["cv"]
53
```

19. user (2026-06-07T09:10:48.163Z)

```
1      """Tuned-segment export for eyeball verification (pure parts; no WSL/GPU)."""
2      import csv
3
4      from backend.eval import export_wasb_segments as ex
5      from backend.tools.rally_slicer import load_rally_labels
```



```

6
7
8 def _write_traj(path):
9     """Synthetic trajectory: two moving segments separated by an invisible gap."""
10    rows = [("Frame", "Visibility", "X", "Y")]
11    for i in range(0, 30):
12        rows.append((i, 1, i * 5, 50))
13    for i in range(30, 50):
14        rows.append((i, 0, -1, -1))
15    for i in range(50, 80):
16        rows.append((i, 1, (i - 50) * 5, 50))
17    with open(path, "w", newline="") as f:
18        csv.writer(f).writerows(rows)
19    return str(path)
20
21
22 def _write_golden(path):
23     with open(path, "w", newline="", encoding="utf-8") as f:
24         w = csv.writer(f)
25         w.writerow(["rally_number", "start_time", "end_time"])
26         w.writerow([1, 0.0, 3.0])
27         w.writerow([2, 5.0, 8.0])
28     return str(path)
29
30
31 def test_seconds_to_mmss():
32     assert ex.seconds_to_mmss(106.031) == "1:46.031"
33     assert ex.seconds_to_mmss(6.031) == "0:06.031"
34     assert ex.seconds_to_mmss(-5) == "0:00.000"
35
36
37 def test_segments_to_rows_numbers_in_time_order():
38     rows = ex.segments_to_rows([(1.0, 3.0), (5.5, 8.25)])
39     assert [r["rally_number"] for r in rows] == ["1", "2"]
40     assert rows[0]["start_time"] == "1.000" and rows[0]["end_time"] == "3.000"
41     assert rows[1]["duration"] == "2.750"
42     assert rows[1]["start_mmss"] == "0:05.500"
43
44
45 def test_exported_csv_feeds_straight_into_the_slicer(tmp_path):
46     """The export schema must be exactly what rally_slicer consumes (the reuse
47 claim)."""
48     out = str(tmp_path / "segs.csv")
49     ex.write_segments_csv(out, [(1.0, 3.0), (5.5, 8.25)])
50     assert load_rally_labels(out) == {"1": (1.0, 3.0), "2": (5.5, 8.25)}
51
52 def test_build_comparison_classifies_match_miss_and_false_positive():
53     preds = [(0.0, 3.0), (5.0, 8.0), (20.0, 22.0)] # 3rd has no golden -> false
positive
54     gts = [(0.1, 3.1), (5.0, 8.0), (10.0, 13.0)] # 3rd has no pred -> miss
55     comp = ex.build_comparison(preds, gts, iou_threshold=0.5)
56     sc = comp["score"]
57     assert (sc.true_positives, sc.false_negatives, sc.false_positives) == (2, 1, 1)
58
59     golden = [r for r in comp["rows"] if r["type"] == "golden"]
60     assert len(golden) == 3
61     assert [r["status"] for r in golden].count("matched") == 2
62     assert [r["status"] for r in golden].count("MISS") == 1
63     extra = [r for r in comp["rows"] if r["type"] == "pred_extra"]
64     assert len(extra) == 1 and extra[0]["status"] == "false_positive"
65     assert extra[0]["start_time"] == "20.000"
66
67
68 def test_run_writes_segments_and_comparison(tmp_path):

```

```

69     traj = _write_traj(tmp_path / "t.csv")
70     gold = _write_golden(tmp_path / "g.csv")
71     out = str(tmp_path / "out" / "segs.csv")
72     res = ex.run(traj, fps=10.0, frame_width=100.0, out_csv=out,
73                 labels_csv=gold, iou=0.5, baseline=True)
74
75     assert res["num_tuned"] >= 1
76     # tuned CSV exists and is slicer-loadable with the reported number of rallies
77     assert len(load_rally_labels(out)) == res["num_tuned"]
78     # baseline + comparison artifacts written
79     assert load_rally_labels(res["baseline_csv"])
80     assert res["comparison"]["num_gt"] == 2
81     with open(res["comparison_csv"], encoding="utf-8") as f:
82         comp_rows = list(csv.DictReader(f))
83     assert any(r["type"] == "golden" for r in comp_rows)
84
85
86     def test_run_min_crossings_gates_low_crossing_segments(tmp_path):
87         # synthetic segments move monotonically in X -> cross the net <2x -> gate(K=2) drops
88         them
89         traj = _write_traj(tmp_path / "t.csv")
90         r0 = ex.run(traj, fps=10.0, frame_width=100.0, out_csv=str(tmp_path / "o0.csv"),
91                     min_crossings=0)
92         r2 = ex.run(traj, fps=10.0, frame_width=100.0, out_csv=str(tmp_path / "o2.csv"),
93                     min_crossings=2)
94         assert r2["num_tuned"] < r0["num_tuned"]
95

```

20. user (2026-06-07T09:10:48.612Z)

```

1     """Pure helpers of the Gemini refine driver (no API calls)."""
2     from backend.eval import gemini_refine as gr
3
4
5     def test_merge_overlaps_unions_overlapping_and_sorts():
6         assert gr.merge_overlaps([(1, 3), (2, 4), (10, 12)]) == [(1, 4), (10, 12)]
7         assert gr.merge_overlaps([(5, 6), (1, 2)]) == [(1, 2), (5, 6)]
8         assert gr.merge_overlaps([]) == []
9
10
11     def test_merge_overlaps_gap_tol_stitches_small_gaps_keeps_real_ones():
12         # 0.5s gap (<= tol) heals; 5s gap (real between-rally) stays separate
13         assert gr.merge_overlaps([(1, 3), (3.5, 5), (10, 12)], gap_tol=1.0) == [(1, 5), (10,
14         12))]
15         # without tolerance the 0.5s gap is preserved (no over-merge)
16         assert gr.merge_overlaps([(1, 3), (3.5, 5)]) == [(1, 3), (3.5, 5)]
17
18     def test_offset_segments_maps_to_full_video_and_drops_invalid():
19         raw = [{"start_time": 1.0, "end_time": 3.0},
20               {"start_time": 2.0, "end_time": 1.0}, # end<=start -> dropped
21               {"nope": 1}] # unparseable -> dropped
22         assert gr._offset_segments(raw, clip_start=10.0, clip_end=20.0) == [(11.0, 13.0)]
23
24
25     def test_offset_segments_clamps_to_clip_bounds():
26         raw = [{"start_time": -5.0, "end_time": 100.0}]
27         assert gr._offset_segments(raw, clip_start=10.0, clip_end=20.0) == [(10.0, 20.0)]
28

```

21. user (2026-06-07T09:10:52.535Z)

```

1     import pytest
2     from unittest.mock import MagicMock, patch
3     from backend.providers.gemini import GeminiProvider

```

```

4
5     def test_gemini_provider_success():
6         # Mock GenAI client
7         with patch("backend.providers.gemini.genai") as mock_genai:
8             client_mock = MagicMock()
9             mock_genai.Client.return_value = client_mock
10
11         # Mock file upload
12         file_mock = MagicMock()
13         file_mock.uri = "gs://dummy"
14         file_mock.name = "dummy_file"
15         file_mock.state.name = "ACTIVE"
16
17         client_mock.files.upload.return_value = file_mock
18         client_mock.files.get.return_value = file_mock
19
20         # Mock generate_content response
21         response_mock = MagicMock()
22         response_mock.text = '["start_time": 1.0, "end_time": 2.0]}'
23         client_mock.models.generate_content.return_value = response_mock
24
25         provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
26         segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
27
28         assert len(segments) == 1
29         assert segments[0]["start_time"] == 1.0
30         assert metrics["upload_time"] >= 0
31         assert metrics["process_time"] >= 0
32         assert metrics["gen_time"] >= 0
33
34         # Ensure delete is called
35         client_mock.files.delete.assert_called_with(name="dummy_file")
36
37     def test_gemini_provider_invalid_json():
38         with patch("backend.providers.gemini.genai") as mock_genai:
39             client_mock = MagicMock()
40             mock_genai.Client.return_value = client_mock
41             file_mock = MagicMock()
42             file_mock.state.name = "ACTIVE"
43             client_mock.files.upload.return_value = file_mock
44             client_mock.files.get.return_value = file_mock
45
46             response_mock = MagicMock()
47             response_mock.text = '{"not_a_list": true}'
48             client_mock.models.generate_content.return_value = response_mock
49
50             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
51             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
52             assert len(segments) == 0
53
54     def test_gemini_provider_rejects_oversize_before_upload(monkeypatch):
55         with patch("backend.providers.gemini.genai") as mock_genai:
56             client_mock = MagicMock()
57             mock_genai.Client.return_value = client_mock
58             # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever uploading
59             monkeypatch.setattr("backend.providers.gemini.os.path.getsize",
60                                 lambda p: 600 * 1024 * 1024)
61             provider = GeminiProvider(api_key="k", model_name="gemini-flash-latest")
62             segments, metrics = provider.analyze_video("big.mp4", "p")
63             assert segments == []
64             client_mock.files.upload.assert_not_called()
65

```

```

66
67     def test_gemini_provider_upload_fail():
68         with patch("backend.providers.gemini.genai") as mock_genai:
69             client_mock = MagicMock()
70             mock_genai.Client.return_value = client_mock
71             client_mock.files.upload.side_effect = Exception("Upload err")
72
73             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
74             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
75             assert len(segments) == 0
76

```

22. user (2026-06-07T09:10:53.122Z)

```

1     """Local-only calibration driver (pure parts; no WSL/GPU/Gemini)."""
2     import csv
3
4     from backend.eval import calibrate_local as cl
5
6
7     def _write_traj(path):
8         rows = [("Frame", "Visibility", "X", "Y")]
9         for i in range(0, 30):
10             rows.append((i, 1, i * 5, 50))      # moving (monotonic in X)
11         for i in range(30, 50):
12             rows.append((i, 0, -1, -1))         # dead time
13         for i in range(50, 80):
14             rows.append((i, 1, (i - 50) * 5, 50))
15         with open(path, "w", newline="") as f:
16             csv.writer(f).writerows(rows)
17         return str(path)
18
19
20     def test_local_grid_size():
21         assert len(cl.local_grid()) == 5 * 3 * 3 * 4    # velocity x merge x min_dur x
min_crossings
22
23
24     def test_predict_fn_returns_windows_and_gate_filters(tmp_path):
25         pf = cl.make_local_predict_fn(_write_traj(tmp_path / "t.csv"), fps=10.0,
frame_width=100.0)
26         no_gate = pf((0.02, 2.0, 1.0, 0))
27         gated = pf((0.02, 2.0, 1.0, 3))            # monotonic motion crosses net <3x ->
gated out
28         assert len(no_gate) >= 1
29         assert len(gated) <= len(no_gate)
30         assert all(e > s for s, e in no_gate)
31

```

23. user (2026-06-07T09:10:53.567Z)

```

1     """Distilled local rally classifier driver (pure parts; no WSL/GPU/Gemini)."""
2     import csv
3
4     from backend.eval import distill_local as dl
5     from backend.eval.classifier import LogisticRegression
6
7
8     def _write_traj(path):
9         rows = [("Frame", "Visibility", "X", "Y")]
10        for i in range(0, 30):
11            rows.append((i, 1, i * 5, 50))
12        for i in range(30, 50):
13            rows.append((i, 0, -1, -1))

```

```

14         for i in range(50, 80):
15             rows.append((i, 1, (i - 50) * 5, 50))
16         with open(path, "w", newline="") as f:
17             csv.writer(f).writerows(rows)
18         return str(path)
19
20
21     def test_build_candidates_feature_shape(tmp_path):
22         intervals, X = dl.build_candidates(_write_traj(tmp_path / "t.csv"), fps=10.0,
frame_width=100.0)
23         assert len(intervals) == len(X) >= 1
24         assert all(len(row) == len(dl.FEATURE_NAMES) for row in X) # 5 invariant features
25         assert all(d > 0 for d, *_ in X) # duration positive
26
27
28     def test_predict_rallies_respects_threshold(tmp_path):
29         intervals, X = dl.build_candidates(_write_traj(tmp_path / "t.csv"), fps=10.0,
frame_width=100.0)
30         y = [1] + [0] * (len(intervals) - 1)
31         model = LogisticRegression(epochs=100).fit(X, y, dl.FEATURE_NAMES)
32         none = dl.predict_rallies(model, X, intervals, threshold=1.01) # impossible cutoff
-> nothing
33         assert none == []
34         some = dl.predict_rallies(model, X, intervals, threshold=0.0) # everything passes
-> merged windows
35         assert len(some) >= 1
36

```

24. user (2026-06-07T09:10:54.407Z)

```

1     """Tests for the GS-4 AutomatedProvider interface stub: registry wiring, the
2     injected-labeler contract, the not-configured guard, the oracle-lineage warning,
3     and the end-to-end path through the GS-2 regression runner."""
4     import logging
5     import pytest
6
7     from backend.annotations import annotation_registry
8     from backend.annotations.base import DONE, FAILED
9     from backend.annotations.automated_provider import (
10         AutomatedProvider, warn_if_same_lineage,
11     )
12
13
14     def _fake_labeler(video_ref, guideline, sport):
15         # Deterministic, model-free stand-in for a real oracle.
16         return [(20.0, 30.0), (0.0, 10.0)] # intentionally unsorted -> provider sorts
17
18
19     # ----- registry -----
20
21     def test_registry_has_auto():
22         assert "auto" in annotation_registry.list_available()
23         p = annotation_registry.get("auto")
24         assert isinstance(p, AutomatedProvider)
25         assert p.name == "auto"
26
27
28     def test_registry_get_injects_labeler_and_lineage():
29         p = annotation_registry.get("auto", labeler=_fake_labeler, lineage="fake")
30         assert p.lineage == "fake"
31         out = p.annotate("vid.mp4")
32         assert out == [(0.0, 10.0), (20.0, 30.0)] # sorted by start
33
34
35     # ----- submit / poll / fetch -----

```

```

36
37 def test_submit_poll_fetch_cycle():
38     p = AutomatedProvider(labeler=_fake_labeler, lineage="fake")
39     job = p.submit("vid.mp4")
40     assert p.poll(job) == DONE
41     assert p.fetch(job) == [(0.0, 10.0), (20.0, 30.0)]
42
43
44 def test_poll_unknown_job_is_failed():
45     p = AutomatedProvider(labeler=_fake_labeler)
46     assert p.poll("never-submitted") == FAILED
47
48
49 def test_fetch_unknown_job_raises():
50     p = AutomatedProvider(labeler=_fake_labeler)
51     with pytest.raises(KeyError):
52         p.fetch("never-submitted")
53
54
55 # ----- the not-configured guard (fail loud, not silent-empty) -----
56
57 def test_unconfigured_labeler_raises():
58     p = annotation_registry.get("auto") # no labeler injected
59     with pytest.raises(NotImplementedError):
60         p.annotate("vid.mp4")
61
62
63 # ----- oracle rule encoded in code -----
64
65 def test_warn_if_same_lineage_true_and_logs(caplog):
66     with caplog.at_level(logging.WARNING):
67         assert warn_if_same_lineage("gemini", "gemini") is True
68         assert any("blind" in r.message.lower() or "lineage" in r.message.lower()
69                 for r in caplog.records)
70
71
72 def test_warn_if_same_lineage_false_when_distinct():
73     assert warn_if_same_lineage("gemini", "actionformer") is False
74     assert warn_if_same_lineage(None, "actionformer") is False
75
76
77 # ----- end-to-end through the GS-2 regression runner -----
78
79 def test_regress_with_auto_tier(tmp_path):
80     from backend.infrastructure.database import Database
81     from backend.eval import regression
82
83     d = Database(str(tmp_path / "t.db"))
84     d.add_video("vid1", "/x/m.mp4", "processed", [])
85     d.add_play_segment("vid1", 0.0, 10.0)
86     d.add_play_segment("vid1", 20.0, 30.0)
87
88     # The 'auto' oracle (fake labeler) supplies ground truth; predictions match ? clean.
89     r = regression.regress_with_provider(
90         d, video_id="vid1", video_ref="vid1", tier="auto",
91         baseline_path=str(tmp_path / "b.json"),
92         provider_kwargs={"labeler": _fake_labeler, "lineage": "fake"},
93     )
94     assert r.precision == 1.0 and r.recall == 1.0
95     assert r.regressed is False
96

```

25. user (2026-06-07T09:10:54.943Z)

```

1     import pytest

```

```

2     from backend.pipeline.segmenters.base import BasePlaySegmenter, SegmenterRegistry
3
4     class DummySegmenter(BasePlaySegmenter):
5         @property
6         def name(self) -> str:
7             return "dummy"
8
9         @property
10        def description(self) -> str:
11            return "Dummy segmenter for testing."
12
13        def process_video(self, video_id, video_path, player_pool=None, max_frames=None):
14            return [{"start_time": 0.0, "end_time": 1.0}]
15
16    def test_registry():
17        registry = SegmenterRegistry()
18        registry.register(DummySegmenter)
19
20        available = registry.list_available()
21        assert "dummy" in available
22        assert available["dummy"] == "Dummy segmenter for testing."
23
24        cls = registry.get("dummy")
25        assert cls == DummySegmenter
26
27        inst = cls(None, {})
28        assert inst.name == "dummy"
29        assert inst.process_video("v1", "path") == [{"start_time": 0.0, "end_time": 1.0}]
30

```

26. user (2026-06-07T09:10:55.460Z)

```

1     """Basic tests for the typed configuration package.
2
3     These ensure:
4     - Defaults are sensible and come from the models.
5     - File overrides work.
6     - Old config files with extra keys are tolerated.
7     - save_default_config produces loadable output.
8     """
9
10    import json
11    import tempfile
12    import os
13
14    import pytest
15
16    from backend.config import load_config, save_default_config, AppConfig
17
18
19    def test_load_default_config():
20        # Force load without the project's config.json so we test the model built-in
21        defaults
22        cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
23        assert isinstance(cfg, AppConfig)
24        assert cfg.sport == "badminton"
25        assert cfg.indexing.default_segmenter == "yolo_hybrid"
26        assert cfg.validation.min_blur_threshold == 20.0 # updated default from review
27        assert cfg.indexing.ai_max_workers == 5
28
29    def test_file_override():
30        with tempfile.TemporaryDirectory() as td:
31            p = os.path.join(td, "override.json")
32            data = {

```

```

33         "indexing": {
34             "default_segmenter": "gemini",
35             "ai_max_workers": 3,
36         },
37         "validation": {
38             "min_blur_threshold": 42.0,
39         },
40     }
41     with open(p, "w") as f:
42         json.dump(data, f)
43
44     cfg = load_config(p)
45     assert cfg.indexing.default_segmenter == "gemini"
46     assert cfg.indexing.ai_max_workers == 3
47     assert cfg.validation.min_blur_threshold == 42.0
48     # Unspecified keys fall back to defaults
49     assert cfg.sport == "badminton"
50
51
52 def test_extra_keys_tolerated():
53     """Old or future keys in config.json must not break loading."""
54     with tempfile.TemporaryDirectory() as td:
55         p = os.path.join(td, "with_extra.json")
56         data = {
57             "sport": "badminton",
58             "indexing": {"default_segmenter": "yolo_hybrid"},
59             "some_future_key": "ignored_for_now",
60             "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
61         }
62         with open(p, "w") as f:
63             json.dump(data, f)
64
65         cfg = load_config(p)
66         assert cfg.indexing.default_segmenter == "yolo_hybrid"
67         assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
68
69
70 def test_save_default_config_roundtrip():
71     with tempfile.TemporaryDirectory() as td:
72         p = os.path.join(td, "saved.json")
73         saved = save_default_config(p)
74         assert str(saved) == p
75         assert os.path.exists(p)
76
77         cfg = load_config(p)
78         assert cfg.indexing.default_segmenter == "yolo_hybrid"
79         assert cfg.validation.min_blur_threshold == 20.0
80
81
82 def test_model_is_the_source_of_truth():
83     """Changing a default in the model should be reflected without touching json."""
84     # This is a documentation test more than anything.
85     cfg = AppConfig()
86     assert cfg.indexing.tracknet.velocity_threshold == 0.02
87
88
89 def test_get_returns_dict_for_nested_sections_legacy_compat():
90     """Regression: legacy consumers (validators/segmenters) do
91     ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
92     .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
93     chained access raises 'X object has no attribute get' on every real run."""
94     cfg = AppConfig()
95     validation = cfg.get("validation", {})
96     assert isinstance(validation, dict)
97     assert validation.get("min_resolution_width") == 1280

```



```

98     # nested-of-nested is also dict-accessible
99     assert isinstance(validation.get("relevance"), dict)
100    assert validation["relevance"].get("court_pixels_min_ratio") == 0.05
101    assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
102    # scalar leaves are returned unchanged; a present scalar ignores the default arg;
103    # a missing key falls back to the default
104    assert cfg.get("ai_provider") == "gemini"
105    assert cfg.get("ai_provider", "ignored-default") == "gemini"
106    assert cfg.get("does_not_exist", "fallback") == "fallback"
107    # leaf keys inside any section resolve via the dynamic section scan (not a hardcoded
108    # list), so a future top-level section's keys would be discoverable too
109    assert cfg.get("min_resolution_width") == 1280 # lives inside `validation`
110    assert cfg.get("begin_padding") == 0.5        # lives inside `stitching`
111

```

27. user (2026-06-07T09:10:55.912Z)

```

1     import pytest
2     import sqlite3
3     import os
4     from backend.infrastructure.database import Database
5
6     def test_database_init(tmp_path):
7         db_path = tmp_path / "test.db"
8         db = Database(str(db_path))
9
10        conn = sqlite3.connect(str(db_path))
11        cursor = conn.cursor()
12        cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
13        tables = [r[0] for r in cursor.fetchall()]
14        assert "videos" in tables
15        assert "play_segments" in tables
16        conn.close()
17
18    def test_database_video(tmp_path):
19        db = Database(str(tmp_path / "test.db"))
20
21        vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
22        assert vid_added is True
23
24        video = db.get_video("vid1")
25        assert video["id"] == "vid1"
26        assert video["filepath"] == "vid1.mp4"
27        assert video["status"] == "pending"
28        assert video["validation_results"] == [{"val": 1}]
29
30        db.clear_video_data("vid1")
31        assert db.get_video("vid1") is None
32
33    def test_database_play_segment(tmp_path):
34        db = Database(str(tmp_path / "test.db"))
35        db.add_video("vid1", "vid1.mp4", "pending", [])
36
37        sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
38        assert sid is not None
39
40        db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
41
42        segments = db.query_play_segments("vid1", {"server": "PlayerA"})
43        assert len(segments) == 1
44        assert segments[0]["id"] == sid
45        assert segments[0]["start_time"] == 10.0
46        assert segments[0]["server"] == "PlayerA"
47        assert segments[0]["metadata"]["foo"] == "bar"
48

```

```

49         assert len(segments[0]["events"]) == 1
50         assert segments[0]["events"][0]["type"] == "smash"
51         assert segments[0]["events"][0]["details"]["speed"] == 300
52
53
54     # --- candidate_segments schema + behavior -----
55
56     # The schema contract: any future change to candidate_segments columns must
57     # consciously update this set, otherwise the test below fails (regression guard).
58     EXPECTED_CANDIDATE_COLUMNS = {
59         "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
60         "duration", "confidence", "stats", "detection_params",
61         "confirmed_segment_id", "human_label", "notes", "created_at",
62     }
63
64
65     def test_candidate_schema_contract(tmp_path):
66         db = Database(str(tmp_path / "test.db"))
67         conn = sqlite3.connect(str(tmp_path / "test.db"))
68         cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
69         conn.close()
70
71         names = {c[1] for c in cols}
72         assert names == EXPECTED_CANDIDATE_COLUMNS
73
74         # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
pk)
75         by_name = {c[1]: c for c in cols}
76         assert by_name["id"][5] == 1 # primary key
77         for col in ("video_id", "detector", "start_time", "end_time", "duration"):
78             assert by_name[col][3] == 1, f"{col} should be NOT NULL"
79
80
81     def test_schema_version_is_current(tmp_path):
82         db_path = tmp_path / "test.db"
83         Database(str(db_path))
84         conn = sqlite3.connect(str(db_path))
85         version = conn.execute("PRAGMA user_version").fetchone()[0]
86         conn.close()
87         assert version == 1
88
89
90     def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
91         db_path = tmp_path / "legacy.db"
92         # Simulate a pre-v1 database that only has the old tables and no version.
93         conn = sqlite3.connect(str(db_path))
94         conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
95         conn.commit()
96         conn.close()
97
98         # Opening with the new Database should add candidate_segments without error...
99         Database(str(db_path))
100        # ...and running init again must be a no-op (idempotent).
101        Database(str(db_path))
102
103        conn = sqlite3.connect(str(db_path))
104        tables = [r[0] for r in conn.execute(
105            "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
106        version = conn.execute("PRAGMA user_version").fetchone()[0]
107        conn.close()
108        assert "candidate_segments" in tables
109        assert version == 1
110
111

```

```

112 def test_candidate_round_trip(tmp_path):
113     db = Database(str(tmp_path / "test.db"))
114     db.add_video("vid1", "vid1.mp4", "processed", [])
115
116     cid = db.add_candidate_segment(
117         "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
118         stats={"peak_velocity": 0.8, "track_id_count": 2},
119         detection_params={"frame_skip": 3},
120     )
121     assert cid is not None
122
123     cands = db.query_candidate_segments("vid1")
124     assert len(cands) == 1
125     c = cands[0]
126     assert c["detector"] == "yolo_hybrid"
127     assert c["start_time"] == 10.0
128     assert c["duration"] == 15.0
129     assert c["confidence"] == 0.42
130     assert c["stats"]["peak_velocity"] == 0.8
131     assert c["detection_params"]["frame_skip"] == 3
132     assert c["confirmed_segment_id"] is None
133
134     # detector filter
135     assert db.query_candidate_segments("vid1", detector="motion") == []
136     assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
137
138
139 def test_candidate_link_and_unlabeled_filter(tmp_path):
140     db = Database(str(tmp_path / "test.db"))
141     db.add_video("vid1", "vid1.mp4", "processed", [])
142
143     cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
144     sid = db.add_play_segment("vid1", 1.0, 4.0)
145
146     assert db.link_candidate_to_segment(cid, sid) is True
147     assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
148
149     # only_unlabeled returns it while human_label is NULL
150     assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
151     with db._get_connection() as conn:
152         conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
153                     ("true_rally", cid))
154         conn.commit()
155     assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
156
157
158 def test_candidate_cascade_on_video_delete(tmp_path):
159     db = Database(str(tmp_path / "test.db"))
160     db.add_video("vid1", "vid1.mp4", "processed", [])
161     db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
162
163     db.clear_video_data("vid1")
164     assert db.query_candidate_segments("vid1") == []
165

```

28. user (2026-06-07T09:10:56.357Z)

```

1     """Tests for the evaluation harness (DB read -> score -> report) and the CLI
2     glue. Uses a real temporary SQLite DB so the query methods are exercised."""
3
4     import json
5     import pytest
6
7     from backend.infrastructure.database import Database
8     from backend.eval.harness import (

```

```

9         evaluate, get_predictions, format_report_text, report_to_dict, _rows_to_intervals,
10     )
11     import run_eval
12
13
14     @pytest.fixture
15     def db(tmp_path):
16         d = Database(str(tmp_path / "test.db"))
17         d.add_video("vid1", "/x/match.mp4", "processed", [])
18         return d
19
20
21     def test_rows_to_intervals_sorts():
22         rows = [{"start_time": 50, "end_time": 60}, {"start_time": 0, "end_time": 10}]
23         assert _rows_to_intervals(rows) == [(0.0, 10.0), (50.0, 60.0)]
24
25
26     def test_get_predictions_final(db):
27         db.add_play_segment("vid1", 0.0, 10.0)
28         db.add_play_segment("vid1", 20.0, 30.0)
29         assert get_predictions(db, "vid1", "final") == [(0.0, 10.0), (20.0, 30.0)]
30
31
32     def test_get_predictions_candidates_with_detector_filter(db):
33         db.add_candidate_segment("vid1", "yolo_hybrid", 0.0, 10.0)
34         db.add_candidate_segment("vid1", "tracknet_hybrid", 40.0, 50.0)
35         yolo = get_predictions(db, "vid1", "candidates", detector="yolo_hybrid")
36         assert yolo == [(0.0, 10.0)]
37         allc = get_predictions(db, "vid1", "candidates")
38         assert len(allc) == 2
39
40
41     def test_get_predictions_unknown_stage_raises(db):
42         with pytest.raises(ValueError):
43             get_predictions(db, "vid1", "bogus")
44
45
46     def test_evaluate_both_stages(db):
47         # GT: two rallies. Candidates catch both; final only confirms one.
48         db.add_candidate_segment("vid1", "yolo_hybrid", 0.0, 10.0)
49         db.add_candidate_segment("vid1", "yolo_hybrid", 20.0, 30.0)
50         db.add_play_segment("vid1", 0.0, 10.0)
51
52         gt = [(0.0, 10.0), (20.0, 30.0)]
53         report = evaluate(db, "vid1", gt, ["candidates", "final"], iou_threshold=0.5)
54
55         cand = report.stages["candidates"].score
56         final = report.stages["final"].score
57         # Candidates find both rallies (detection is fine)...
58         assert cand.recall == 1.0
59         # ...but final only confirms one (segmentation/handoff dropped the other).
60         assert final.recall == pytest.approx(0.5)
61         assert final.true_positives == 1
62
63
64     def test_evaluate_with_pr_curve(db):
65         db.add_play_segment("vid1", 0.0, 10.0)
66         report = evaluate(db, "vid1", [(0.0, 10.0)], ["final"], with_pr_curve=True)
67         assert len(report.stages["final"].pr_curve) == 9
68
69
70     def test_format_report_text_contains_stage_rows(db):
71         db.add_play_segment("vid1", 0.0, 10.0)
72         report = evaluate(db, "vid1", [(0.0, 10.0)], ["final"])
73         text = format_report_text(report)

```

```

74     assert "vid1" in text
75     assert "final" in text
76
77
78     def test_format_report_failures_lists_misses(db):
79         # GT has a rally with no matching prediction -> should appear as a miss.
80         report = evaluate(db, "vid1", [(0.0, 10.0)], ["final"])
81         text = format_report_text(report, show_failures=True)
82         assert "missed rallies" in text
83         assert "00:00:00" in text # the missed rally's start timestamp
84
85
86     def test_report_to_dict_is_json_serialisable(db):
87         db.add_play_segment("vid1", 0.0, 10.0)
88         report = evaluate(db, "vid1", [(0.0, 10.0)], ["final"], with_pr_curve=True)
89         d = report_to_dict(report)
90         json.dumps(d) # must not raise
91         assert d["video_id"] == "vid1"
92         assert d["stages"]["final"]["score"]["f1"] == 1.0
93
94
95     # ----- CLI glue
96
97     def test_cli_scaffold(tmp_path, capsys):
98         ann = str(tmp_path / "rallies.csv")
99         rc = run_eval.main(["--scaffold", "--annotations", ann])
100        assert rc == 0
101        from backend.eval.annotations import load_annotations
102        assert load_annotations(ann) == [] # header-only template
103
104
105     def test_cli_requires_annotations(capsys):
106         assert run_eval.main([]) == 2
107
108
109     def test_cli_missing_db_errors(tmp_path):
110         ann = str(tmp_path / "r.csv")
111         from backend.eval.annotations import write_scaffold
112         write_scaffold(ann)
113         rc = run_eval.main(["--video-id", "abc", "--annotations", ann,
114                             "--db", str(tmp_path / "nope.db")])
115         assert rc == 2
116
117
118     def test_cli_full_run(tmp_path, capsys):
119         db_path = str(tmp_path / "sports.db")
120         d = Database(db_path)
121         d.add_video("vid1", "/x/m.mp4", "processed", [])
122         d.add_play_segment("vid1", 0.0, 10.0)
123
124         ann = tmp_path / "r.csv"
125         ann.write_text("start,end\n0,10\n")
126
127         json_out = str(tmp_path / "out.json")
128         rc = run_eval.main(["--video-id", "vid1", "--annotations", str(ann),
129                             "--db", db_path, "--stage", "final", "--json", json_out])
130         assert rc == 0
131         captured = capsys.readouterr().out
132         assert "Rally evaluation" in captured
133         with open(json_out) as f:
134             report = json.load(f)
135         assert report["stages"]["final"]["score"]["f1"] == 1.0
136
137
138     def test_get_file_md5_stable(tmp_path):

```

```

139     f = tmp_path / "v.mp4"
140     f.write_bytes(b"hello world")
141     h1 = run_eval.get_file_md5(str(f))
142     h2 = run_eval.get_file_md5(str(f))
143     assert h1 == h2 and len(h1) == 32
144

```

29. user (2026-06-07T09:10:59.367Z)

```

1     """Tests for the regression runner (GS-2): baseline store, regress(), the
2     provider-driven end-to-end path, and the CLI exit-code contract."""
3     import json
4     import pytest
5
6     from backend.infrastructure.database import Database
7     from backend.eval import regression
8     import run_regression
9
10
11     @pytest.fixture
12     def db(tmp_path):
13         d = Database(str(tmp_path / "t.db"))
14         d.add_video("vid1", "/x/m.mp4", "processed", [])
15         # Two confirmed segments matching the GT below ? perfect baseline.
16         d.add_play_segment("vid1", 0.0, 10.0)
17         d.add_play_segment("vid1", 20.0, 30.0)
18         return d
19
20
21     GT = [(0.0, 10.0), (20.0, 30.0)]
22
23
24     # ----- baseline store -----
25
26     def test_save_and_load_baseline_roundtrip(tmp_path):
27         p = str(tmp_path / "b.json")
28         regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
29         b = regression.load_baselines(p)
30         assert b["vid1::final"]["precision"] == 1.0
31
32
33     def test_load_baselines_absent_is_empty(tmp_path):
34         assert regression.load_baselines(str(tmp_path / "nope.json")) == {}
35
36
37     # ----- regress() -----
38
39     def test_no_baseline_is_not_a_regression(db, tmp_path):
40         r = regression.regress(db, "vid1", GT, baseline_path=str(tmp_path / "b.json"))
41         assert r.regressed is False
42         assert r.precision == 1.0 and r.recall == 1.0
43         assert any("no baseline" in why for why in r.reasons)
44
45
46     def test_matching_baseline_passes(db, tmp_path):
47         p = str(tmp_path / "b.json")
48         regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
49         r = regression.regress(db, "vid1", GT, baseline_path=p)
50         assert r.regressed is False
51
52
53     def test_recall_drop_beyond_tolerance_regresses(tmp_path):
54         # DB has only ONE of the two GT rallies ? recall 0.5 vs baseline 1.0.
55         d = Database(str(tmp_path / "t.db"))
56         d.add_video("vid1", "/x/m.mp4", "processed", [])

```

```

57     d.add_play_segment("vid1", 0.0, 10.0)
58     p = str(tmp_path / "b.json")
59     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
60
61     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
62     assert r.regressed is True
63     assert r.recall == pytest.approx(0.5)
64     assert any("recall" in why for why in r.reasons)
65
66
67 def test_within_tolerance_does_not_regress(tmp_path):
68     d = Database(str(tmp_path / "t.db"))
69     d.add_video("vid1", "/x/m.mp4", "processed", [])
70     d.add_play_segment("vid1", 0.0, 10.0)
71     d.add_play_segment("vid1", 20.0, 30.0)
72     p = str(tmp_path / "b.json")
73     # Baseline only 0.02 above current perfect score ? within default 0.05 tolerance.
74     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
75     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
76     assert r.regressed is False
77
78
79 # ----- provider-driven end-to-end (uses the GS-1 FileProvider) -----
80
81 def test_regress_with_provider_file_tier(db, tmp_path):
82     csv = tmp_path / "gt.csv"
83     csv.write_text("start,end\n0,10\n20,30\n")
84     r = regression.regress_with_provider(
85         db, video_id="vid1", video_ref=str(csv), tier="file",
86         baseline_path=str(tmp_path / "b.json"),
87     )
88     assert r.precision == 1.0 and r.recall == 1.0
89
90
91 # ----- CLI exit-code contract (CI gating) -----
92
93 def test_cli_update_baseline_then_clean_run(db, tmp_path, capsys, monkeypatch):
94     # Point the runner at our temp DB via --db; CSV ground truth via 'file' tier.
95     csv = tmp_path / "gt.csv"
96     csv.write_text("start,end\n0,10\n20,30\n")
97     base = str(tmp_path / "b.json")
98     db_path = db.db_path if hasattr(db, "db_path") else str(tmp_path / "t.db")
99
100     # First, snapshot the baseline ? exit 0.
101     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
102                             "--annotations", str(csv), "--db", db_path,
103                             "--baseline", base, "--update-baseline"])
104     assert rc == 0
105     assert "REGRESSED" not in capsys.readouterr().out
106
107
108 def test_cli_regression_exits_nonzero(tmp_path, capsys):
109     # DB missing one rally vs a baseline of perfect ? regression ? exit 1.
110     d = Database(str(tmp_path / "t.db"))
111     d.add_video("vid1", "/x/m.mp4", "processed", [])
112     d.add_play_segment("vid1", 0.0, 10.0)
113     csv = tmp_path / "gt.csv"
114     csv.write_text("start,end\n0,10\n20,30\n")
115     base = str(tmp_path / "b.json")
116     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
117
118     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
119                             "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
120                             "--baseline", base])
121     assert rc == 1

```

```
122         assert "REGRESSED" in capsys.readouterr().out
123
```

30. user (2026-06-07T09:11:00.310Z)

```
1     import pytest
2     from unittest.mock import MagicMock, patch
3     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4
5     @patch("backend.pipeline.segmenters.gemini.provider_registry")
6     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
7     @patch("os.environ.get")
8     def test_gemini_segmenter_single_file(mock_env_get, mock_get_dur,
mock_provider_registry):
9         mock_env_get.return_value = "dummy_api_key"
10        mock_get_dur.return_value = 100.0 # 100s video
11
12        mock_provider = MagicMock()
13        mock_provider.analyze_video.return_value = ([
14            {"start_time": 10.0, "end_time": 20.0, "shuttle_exchange_count": 5}
15        ], {"upload_time": 1, "process_time": 1, "gen_time": 1})
16        mock_provider_registry.get.return_value = mock_provider
17
18        db_mock = MagicMock()
19        db_mock.add_play_segment.return_value = "segment_1"
20
21        config = {
22            "indexing": {
23                "chunk_duration": 300, # Large chunk duration, falls back to single file
24                "chunk_overlap": 30
25            }
26        }
27
28        segmenter = GeminiPlaySegmenter(db_mock, config)
29        res = segmenter.process_video("v1", "dummy.mp4")
30
31        assert len(res) == 1
32        assert res[0]["id"] == "segment_1"
33        assert res[0]["start_time"] == 10.0
34        assert res[0]["end_time"] == 20.0
35        assert res[0]["metadata"]["shot_count"] == 5
36
37        @patch("backend.pipeline.segmenters.gemini.provider_registry")
38        @patch("backend.pipeline.segmenters.gemini.get_video_duration")
39        @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
40        @patch("os.environ.get")
41        def test_gemini_segmenter_chunked(mock_env_get, mock_extract, mock_get_dur,
mock_provider_registry):
42            mock_env_get.return_value = "dummy_api_key"
43            mock_get_dur.return_value = 200.0 # 200s video
44            mock_extract.return_value = True
45
46            mock_provider = MagicMock()
47            # Chunk 1 returns 1 segment, Chunk 2 returns another
48            def analyze_video_side_effect(path, prompt, chunk_duration, label, **kwargs):
49                if "Chunk 1" in label:
50                    return [{"start_time": 10.0, "end_time": 20.0}], {}
51                else:
52                    return [{"start_time": 30.0, "end_time": 40.0}], {}
53
54            mock_provider.analyze_video.side_effect = analyze_video_side_effect
55            mock_provider_registry.get.return_value = mock_provider
56
57            db_mock = MagicMock()
58            db_mock.add_play_segment.side_effect = ["segment_1", "segment_2"]
```



```

59
60     config = {
61         "indexing": {
62             "chunk_duration": 150, # Smaller than video, causes chunking
63             "chunk_overlap": 10
64         }
65     }
66
67     segmenter = GeminiPlaySegmenter(db_mock, config)
68     res = segmenter.process_video("v1", "dummy.mp4")
69
70     # Should be 2 segments
71     assert len(res) == 2
72
73     # 1st chunk start is 0
74     assert res[0]["start_time"] == 10.0
75     # 2nd chunk start is 150-10 = 140
76     # So its local time 30 -> 140+30 = 170.0
77     # Let's just check length, exact timing requires checking array order (which may
vary due to concurrent futures)
78     times = [r["start_time"] for r in res]
79     assert 10.0 in times
80     assert 170.0 in times
81

```

31. user (2026-06-07T09:11:00.799Z)

```

1     """Audio hit detector (pure DSP; no ffmpeg/video). Synthetic planted transients."""
2     import numpy as np
3
4     from backend.pipeline.audio import hit_detector as hd
5     from backend.pipeline.audio.hit_detector import HitEvent
6     from backend.eval.audio.e1_probe import cluster_hits
7
8
9     def _synth(sr=16000, dur=6.0, times=(0.5, 1.2, 2.0, 3.1, 4.4, 5.0), seed=0):
10         rng = np.random.default_rng(seed)
11         x = (rng.standard_normal(int(sr * dur)) * 0.002).astype(np.float32) # quiet
background
12         burst = int(0.005 * sr) # 5ms
transient
13         planted = []
14         for t in times:
15             i = int(t * sr)
16             if i + burst <= len(x): # skip out-
of-range
17                 x[i:i + burst] += rng.standard_normal(burst).astype(np.float32) * 0.5 #
loud "thwack"
18                 planted.append(t)
19         return x, sr, tuple(planted)
20
21
22     def test_detect_hits_finds_planted_transients():
23         x, sr, times = _synth()
24         hits = hd.detect_hits(x, sr, k=2.5)
25         detected = [h.t for h in hits]
26         assert len(hits) >= len(times) - 1 # find most planted hits
27         for t in times:
28             assert any(abs(t - d) < 0.06 for d in detected), f"missed planted hit @{t}s; got
{detected}"
29
30
31     def test_detect_hits_empty_on_silence():
32         assert hd.detect_hits(np.zeros(16000 * 2, dtype=np.float32), 16000) == []
33

```

```

34
35 def test_moving_stats_shape_and_values():
36     env = np.ones(50, dtype=float)
37     m, s = hd._moving_stats(env, 11)
38     assert len(m) == len(env) == len(s)
39     assert np.allclose(m, 1.0) and np.allclose(s, 0.0) # constant signal: mean 1,
std 0
40
41
42 def test_onset_envelope_length_aligns_to_frames():
43     x, sr, _ = _synth(dur=2.0)
44     env, hop_sec = hd.onset_envelope(x, sr, frame=1024, hop=256)
45     assert hop_sec == 256 / sr
46     assert len(env) == 1 + (len(x) - 1024) // 256
47
48
49 def test_onset_envelope_band_limit_changes_signal_same_length():
50     x, sr, _ = _synth(dur=3.0)
51     full, _ = hd.onset_envelope(x, sr)
52     banded, _ = hd.onset_envelope(x, sr, band=(2000, 7000))
53     assert len(full) == len(banded)
54     assert not np.allclose(full, banded) # band-limiting changes
the flux
55
56
57 def test_detect_hits_with_band_still_finds_broadband_clicks():
58     x, sr, times = _synth()
59     hits = hd.detect_hits(x, sr, k=2.5, band=(1000, 7000)) # clicks are broadband ->
still found
60     detected = [h.t for h in hits]
61     assert len(hits) >= len(times) - 1
62     for t in times:
63         assert any(abs(t - d) < 0.06 for d in detected)
64
65
66 def test_cluster_hits_drops_lone_keeps_cluster():
67     hits = [HitEvent(10.0, 1.0)] + [HitEvent(t, 1.0) for t in (30.0, 30.6, 31.3, 32.0)]
68     cl = cluster_hits(hits, gap_sec=2.0, min_hits=2)
69     assert len(cl) == 1 # lone hit @10 dropped
70     s, e = cl[0]
71     assert s < 30.0 and e > 32.0 # cluster span (with pad)
72

```

32. user (2026-06-07T09:11:01.266Z)

```

1     """Shuttle net-crossing rally gate (pure; no WSL/GPU)."""
2     from backend.pipeline.detectors.tracknet_runner import TrajectoryPoint
3     from backend.pipeline.detectors import rally_gate as rg
4
5
6     def P(frame, x, y, visible=True):
7         return TrajectoryPoint(frame=frame, visible=visible, x=x, y=y)
8
9
10    def test_estimate_play_axis_picks_larger_spread():
11        pts = [P(i, 500 + (i % 2), 50 if i % 2 else 350) for i in range(20)] # y varies, x
~const
12        axis, net, span = rg.estimate_play_axis(pts)
13        assert axis == "y"
14        assert span > 100
15
16
17    def test_count_net_crossings_counts_sign_changes():
18        pts = [P(0, 0, 10), P(1, 0, 200), P(2, 0, 10), P(3, 0, 200)] #
below/above/below/above

```

```

19         assert rg.count_net_crossings(pts, "y", net=100.0, deadband=5.0) == 3
20
21
22     def test_deadband_ignores_jitter_near_net():
23         pts = [P(0, 0, 101), P(1, 0, 99), P(2, 0, 102), P(3, 0, 98)] # all within deadband
of net
24         assert rg.count_net_crossings(pts, "y", net=100.0, deadband=10.0) == 0
25
26
27     def test_invisible_points_are_skipped():
28         pts = [P(0, 0, 10), P(1, 0, 200, visible=False), P(2, 0, 10)] # invisible middle ->
no flip
29         assert rg.count_net_crossings(pts, "y", net=100.0, deadband=5.0) == 0
30
31
32     def test_gate_kills_single_toss_keeps_multi_crossing_rally():
33         fps = 10.0
34         rally = [P(i, 500, 50 if i % 2 == 0 else 250) for i in range(0, 10)] # alternates
-> many crossings
35         toss = [P(20, 500, 50), P(25, 500, 250)] # one
crossing
36         pts = rally + toss
37         windows = [(0.0, 1.0), (2.0, 3.0)]
38         gated = rg.gate_windows(pts, windows, fps, min_crossings=2)
39         assert gated[0].kept and gated[0].crossings >= 2
40         assert not gated[1].kept and gated[1].crossings <= 1
41         assert rg.apply_gate(pts, windows, fps, min_crossings=2) == [(0.0, 1.0)]
42

```

33. user (2026-06-07T09:11:02.209Z)

```

1     """Tests for the indexer observability harvest + footgun rules.
2
3     These exercise the broken-run scenarios the report is meant to make debuggable.
4     obsreport is a soft dependency; skip cleanly if it isn't installed.
5     """
6     import os
7
8     import pytest
9
10    pytest.importorskip("obsreport")
11
12    from backend.reporting import emit_indexer_report, harvest # noqa: E402
13    from backend.pipeline.validators.base import ValidationResult # noqa: E402
14
15
16    class _FakeDB:
17        def __init__(self, play=None, candidates=None, existing=False):
18            self._play = play or []
19            self._candidates = candidates or []
20            self._existing = existing
21
22        def query_play_segments(self, video_id, filters):
23            return list(self._play)
24
25        def query_candidate_segments(self, video_id, detector=None, only_unlabeled=False):
26            return list(self._candidates)
27
28        def get_video(self, video_id):
29            return {"id": video_id} if self._existing else None
30
31
32    def _vr(name, passed, message="ok"):
33        return ValidationResult(validator_name=name, passed=passed, message=message)
34

```

```

35
36     _FFPROBE_OK = {
37         "streams": [{"width": 1920, "height": 1080, "r_frame_rate": "60000/1001",
"codec_name": "hevc"}],
38         "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "5500000000"},
39     }
40
41
42     def _record(**kw):
43         base = dict(
44             video_id="md5abc",
45             video_path=None,
46             config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing":
{"default_segmenter": "wasb_hybrid"}},
47             val_results=[_vr("format_check", True), _vr("relevance_check", True)],
48             val_context={"ffprobe_meta": _FFPROBE_OK},
49             play_segments=[{"duration": 5.0, "metadata": {"detector": "wasb-hybrid-
gemini"}}],
50             candidates=[{"id": 1}, {"id": 2}],
51             segmenter_requested="wasb_hybrid",
52             segmenter_actual="wasb_hybrid",
53             was_reingest=False,
54             segmentation_ran=True,
55         )
56         base.update(kw)
57         return harvest.record_run(**base).envelope
58
59
60     def _codes(env):
61         return {f.code for f in env.flags}
62
63
64     def test_healthy_run_is_clean(monkeypatch):
65         monkeypatch.setenv("GEMINI_API_KEY", "x")
66         env = _record()
67         assert env.verdict == "pass"
68         assert _codes(env) == set()
69         assert env.video_meta.fps == 59.94
70         assert env.video_meta.fps_source == "probed"
71         seg = env.get_stage("segmentation")
72         assert {m.key: m.value for m in seg.metrics}["segment_count"] == 1
73         assert {m.key: m.value for m in seg.metrics}["candidate_windows"] == 2
74         assert env.lint() == []
75
76
77     def test_silent_provider_noop_fires_when_no_key_and_zero_segments(monkeypatch):
78         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
79         env = _record(play_segments=[])
80         assert "silent_provider_noop" in _codes(env)
81         assert env.verdict == "fail"
82         flag = next(f for f in env.flags if f.code == "silent_provider_noop")
83         assert flag.faq_ref == "README#Q7"
84
85
86     def test_ai_empty_warns_when_key_present_but_zero_segments(monkeypatch):
87         monkeypatch.setenv("GEMINI_API_KEY", "x")
88         env = _record(play_segments=[])
89         codes = _codes(env)
90         assert "ai_empty_vs_no_rallies" in codes
91         assert "silent_provider_noop" not in codes
92         assert env.verdict == "pass_with_warnings"
93
94
95     def test_motion_segmenter_flags_synthetic_metadata(monkeypatch):
96         monkeypatch.delenv("GEMINI_API_KEY", raising=False)

```

```

97     env = _record(segmenter_requested="motion", segmenter_actual="motion",
98                   config={"indexing": {"default_segmenter": "motion"}})
99     codes = _codes(env)
100    assert "synthetic_motion_metadata" in codes
101    # motion is not AI-based -> the provider-noop rule must NOT fire even with no key
102    assert "silent_provider_noop" not in codes
103
104
105    def test_segmenter_divergence_fires_only_without_explicit_override(monkeypatch):
106        monkeypatch.setenv("GEMINI_API_KEY", "x")
107        cfg = {"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}}
108        # No explicit request, but a different segmenter ran -> a fallback diverged: flag
109        it.
110        env = _record(segmenter_requested=None, segmenter_actual="gemini", config=cfg)
111        assert "segmenter_divergence" in _codes(env)
112        # Explicit --segmenter override that differs from config default is NOT a
113        divergence.
114        env2 = _record(segmenter_requested="gemini", segmenter_actual="gemini", config=cfg)
115        assert "segmenter_divergence" not in _codes(env2)
116
117    def test_reingest_info_flag(monkeypatch):
118        monkeypatch.setenv("GEMINI_API_KEY", "x")
119        env = _record(was_reingest=True)
120        assert "reingest_replaced_prior" in _codes(env)
121
122    def test_validation_failure_halts_and_flags(monkeypatch):
123        monkeypatch.setenv("GEMINI_API_KEY", "x")
124        env = _record(
125            val_results=[_vr("format_check", True), _vr("relevance_check", False, "court
126            color not found")],
127            play_segments=[],
128            segmenter_actual=None,
129            segmentation_ran=False,
130        )
131        assert env.get_stage("validation").status == "error"
132        assert env.get_stage("segmentation").status == "not_run"
133        assert "validation_failed" in _codes(env)
134        assert env.verdict == "fail"
135
136    def test_fps_fallback_flag_when_no_probe_but_segmented(monkeypatch):
137        monkeypatch.setenv("GEMINI_API_KEY", "x")
138        env = _record(val_context={}, segmentation_ran=True)
139        assert env.video_meta.fps_source == "fallback"
140        assert "fps_fallback_used" in _codes(env)
141
142
143    def test_emit_writes_files_next_to_video(tmp_path, monkeypatch):
144        monkeypatch.setenv("GEMINI_API_KEY", "x")
145        video = tmp_path / "match.mp4"
146        video.write_bytes(b"not really a video")
147        db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
148        candidates=[{"id": 1}])
149        paths = emit_indexer_report(
150            db=db, video_id="md5abc", video_path=str(video),
151            config={"ai_provider": "gemini", "indexing": {"default_segmenter":
152            "wasb_hybrid"}}},
153            val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
154            _FFPROBE_OK},
155            segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
156            was_reingest=False, segmentation_ran=True,
157        )
158        assert paths is not None

```

```

156     assert os.path.exists(paths["json"])
157     assert (tmp_path / "match.report.md").exists()
158     assert (tmp_path / "match.summary.md").exists()
159
160
161     def test_emit_accepts_typed_appconfig(tmp_path, monkeypatch):
162         """Reporting must work whether main passes a plain dict or the typed AppConfig
163         (backend.config) ? the reporting layer normalizes AppConfig -> dict."""
164         from backend.config import AppConfig
165
166         monkeypatch.setenv("GEMINI_API_KEY", "x")
167         video = tmp_path / "m.mp4"
168         video.write_bytes(b"x")
169         db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
candidates=[{"id": 1}])
170         cfg = AppConfig() # typed config object, not a dict
171         paths = emit_indexer_report(
172             db=db, video_id="md5cfg", video_path=str(video), config=cfg,
173             val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
_FFPROBE_OK},
174             segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
175             was_reingest=False, segmentation_ran=True,
176         )
177         assert paths is not None
178         assert os.path.exists(paths["json"])
179
180
181     def test_emit_disabled_via_config(tmp_path, monkeypatch):
182         monkeypatch.setenv("GEMINI_API_KEY", "x")
183         db = _FakeDB()
184         paths = emit_indexer_report(
185             db=db, video_id="md5abc", video_path=str(tmp_path / "m.mp4"),
186             config={"report": {"enabled": False}},
187             val_results=[_vr("format_check", True)], val_context={},
188             segmenter_actual="motion", segmentation_ran=True,
189         )
190         assert paths is None
191

```

34. user (2026-06-07T09:11:02.677Z)

```

1     """Tests for the TrackNetHybridSegmenter pipeline. The WSL/TensorFlow runner is
2     mocked out (TrackNetRunner), so these run anywhere ? they exercise Phase 1-4 glue:
3     healthcheck gating, candidate persistence, AI handoff, and DB population/linking."""
4
5     import cv2
6     import pytest
7     from unittest.mock import MagicMock, patch
8
9     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
10
11
12     def _window(start=1.0, end=4.0):
13         return {
14             "start": start, "end": end, "active_frame_count": 30,
15             "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1,
16             "chunk_index": 0,
17         }
18
19
20     def _patched(fn):
21         """Stack the patches every pipeline test needs. fps/width probing is stubbed
22         so cv2 internals don't need mocking (covered separately).
23
24         Patches are applied innermost-first, and mock.patch injects the innermost

```

```

25     patch as the *first* positional arg ? so this order matches the signature
26     (mock_runner_cls, mock_probe, mock_dur, mock_extract, mock_provider_registry,
27     mock_env)."""
28     fn = patch("backend.pipeline.segmenters.tracknet_hybrid.TrackNetRunner")(fn)
29     fn = patch.object(TrackNetHybridSegmenter, "_probe_fps_width", return_value=(30.0,
1280.0))(fn)
30     fn = patch("backend.pipeline.segmenters.tracknet_hybrid.get_video_duration")(fn)
31     fn = patch("backend.pipeline.segmenters.tracknet_hybrid.extract_video_chunk")(fn)
32     fn = patch("backend.pipeline.segmenters.tracknet_hybrid.provider_registry")(fn)
33     fn = patch("os.environ.get")(fn)
34     return fn
35
36
37     def _make_runner(windows=None, csv="out.csv", healthy=True):
38         runner = MagicMock()
39         runner.healthcheck.return_value = (healthy, "TF 2.17 GPUs 1" if healthy else "no
GPU")
40         runner.run_predict.return_value = csv
41         runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
42         runner.trajectory_to_action_windows.return_value = windows if windows is not None
else [_window()]
43         return runner
44
45
46     @_patched
47     def test_pipeline_happy_path(mock_runner_cls, mock_probe, mock_dur, mock_extract,
48                                mock_provider_registry, mock_env):
49         mock_env.return_value = "dummy_key"
50         mock_dur.return_value = 30.0
51         mock_extract.return_value = True
52         mock_runner_cls.return_value = _make_runner()
53
54         provider = MagicMock()
55         provider.analyze_video.return_value = (
56             [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}], {})
57         mock_provider_registry.get.return_value = provider
58
59         db = MagicMock()
60         db.add_candidate_segment.return_value = 55
61         db.add_play_segment.return_value = 200
62
63         seg = TrackNetHybridSegmenter(db, {"indexing": {}})
64         res = seg.process_video("v1", "clip.mp4")
65
66         assert len(res) == 1
67         assert res[0]["id"] == 200
68         # Candidate persisted under the tracknet detector name...
69         _, kwargs = db.add_candidate_segment.call_args
70         assert kwargs["detector"] == "tracknet_hybrid"
71         # ...and the confirmed segment linked back to it.
72         db.link_candidate_to_segment.assert_called_once_with(55, 200)
73
74
75     @_patched
76     def test_pipeline_no_api_key_returns_empty(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
77                                               mock_provider_registry, mock_env):
78         mock_env.return_value = None
79         mock_dur.return_value = 30.0
80         mock_runner_cls.return_value = _make_runner()
81
82         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
83         assert seg.process_video("v1", "clip.mp4") == []
84
85

```

```

86     @_patched
87     def test_pipeline_healthcheck_failure_aborts(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
88                                     mock_provider_registry, mock_env):
89         mock_env.return_value = "dummy_key"
90         mock_dur.return_value = 30.0
91         mock_extract.return_value = True
92         mock_runner_cls.return_value = _make_runner(healthy=False)
93         mock_provider_registry.get.return_value = MagicMock()
94
95         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
96         res = seg.process_video("v1", "clip.mp4")
97         assert res == []
98         # Should bail before ever running inference.
99         mock_runner_cls.return_value.run_predict.assert_not_called()
100
101
102     @_patched
103     def test_pipeline_no_trajectory_aborts(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
104                                     mock_provider_registry, mock_env):
105         mock_env.return_value = "dummy_key"
106         mock_dur.return_value = 30.0
107         mock_runner_cls.return_value = _make_runner(csv=None) # predict produced nothing
108         mock_provider_registry.get.return_value = MagicMock()
109
110         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
111         assert seg.process_video("v1", "clip.mp4") == []
112
113
114     @_patched
115     def test_pipeline_store_candidates_disabled(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
116                                     mock_provider_registry, mock_env):
117         mock_env.return_value = "dummy_key"
118         mock_dur.return_value = 30.0
119         mock_extract.return_value = True
120         mock_runner_cls.return_value = _make_runner()
121
122         provider = MagicMock()
123         provider.analyze_video.return_value = ([{"start_time": 0.5, "end_time": 3.0}], {})
124         mock_provider_registry.get.return_value = provider
125
126         db = MagicMock()
127         db.add_play_segment.return_value = 200
128
129         seg = TrackNetHybridSegmenter(db, {"indexing": {"store_yolo_candidates": False}})
130         seg.process_video("v1", "clip.mp4")
131
132         db.add_candidate_segment.assert_not_called()
133         db.link_candidate_to_segment.assert_not_called()
134
135
136     @_patched
137     def test_pipeline_drops_unparseable_segments(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
138                                     mock_provider_registry, mock_env):
139         mock_env.return_value = "dummy_key"
140         mock_dur.return_value = 30.0
141         mock_extract.return_value = True
142         mock_runner_cls.return_value = _make_runner()
143
144         provider = MagicMock()
145         provider.analyze_video.return_value = (
146             [{"start_time": 0.5, "end_time": 3.0}, {"start_time": "N/A", "end_time": "??"}],

```



```

{ })
147         mock_provider_registry.get.return_value = provider
148
149         db = MagicMock()
150         db.add_candidate_segment.return_value = 55
151         db.add_play_segment.return_value = 200
152
153         seg = TrackNetHybridSegmenter(db, {"indexing": {}})
154         res = seg.process_video("v1", "clip.mp4")
155         assert len(res) == 1 # the "N/A" segment is dropped
156
157
158     def test_probe_fps_width_fallbacks():
159         with patch("backend.pipeline.segmenters.tracknet_hybrid.cv2.VideoCapture") as
mock_cap:
160             bad = MagicMock()
161             bad.isOpened.return_value = False
162             bad.get.return_value = 0.0
163             mock_cap.return_value = bad
164             fps, width = TrackNetHybridSegmenter._probe_fps_width("missing.mp4")
165             assert fps == 30.0 and width == 1280.0
166
167
168     def test_segementer_identity():
169         seg = TrackNetHybridSegmenter(MagicMock(), {})
170         assert seg.name == "tracknet_hybrid"
171         assert "TrackNet" in seg.description
172

```

35. user (2026-06-07T09:11:03.119Z)

```

1     """Tests for the TrackNet runner's pure-Python pieces (path translation, CSV
2     parsing, trajectory -> action-window conversion). The WSL subprocess invocation
3     itself is not exercised here ? these run anywhere, no WSL/TensorFlow needed."""
4
5     import os
6     import subprocess
7     import textwrap
8     from unittest.mock import patch
9
10    from backend.pipeline.detectors.tracknet_runner import (
11        TrackNetRunner,
12        TrackNetConfig,
13        TrajectoryPoint,
14        to_wsl_mnt_path,
15    )
16
17
18    def _proc(returncode=0, stdout="", stderr=""):
19        return subprocess.CompletedProcess(args=[], returncode=returncode,
20                                           stdout=stdout, stderr=stderr)
21
22
23    def test_to_wsl_mnt_path_windows():
24        assert to_wsl_mnt_path(r"C:\Users\avidu\v.mp4") == "/mnt/c/Users/avidu/v.mp4"
25        assert to_wsl_mnt_path(r"D:\clips\m.mp4") == "/mnt/d/clips/m.mp4"
26
27
28    def test_to_wsl_mnt_path_passthrough():
29        # Already-POSIX paths are left alone.
30        assert to_wsl_mnt_path("/home/avidullu/clips/v.mp4") == "/home/avidullu/clips/v.mp4"
31        assert to_wsl_mnt_path("~/clips/v.mp4") == "~/clips/v.mp4"
32
33
34    def test_config_from_indexing_cfg():

```

```

35     cfg = TrackNetConfig.from_indexing_cfg({"tracknet": {
36         "wsl_distro": "Ubuntu-22.04", "conda_env": "TN", "weights_path": "/w/m.h5",
37         "queue_length": 7, "stage_in_wsl": False,
38     }})
39     assert cfg.distro == "Ubuntu-22.04"
40     assert cfg.conda_env == "TN"
41     assert cfg.weights_path == "/w/m.h5"
42     assert cfg.queue_length == 7
43     assert cfg.stage_in_wsl is False
44
45
46 def test_config_defaults_when_absent():
47     cfg = TrackNetConfig.from_indexing_cfg({})
48     assert cfg.distro == "Ubuntu"
49     assert cfg.conda_env == "TrackNetV4"
50     assert cfg.weights_path == "" # must be supplied by the user
51
52
53 def test_parse_trajectory_csv(tmp_path):
54     csv_file = tmp_path / "v_predict.csv"
55     csv_file.write_text(textwrap.dedent("""\
56         Frame,Visibility,X,Y
57         0,0,0,0
58         1,1,100,200
59         2,1,140,210
60         3,0,0,0
61     """))
62     pts = TrackNetRunner.parse_trajectory_csv(str(csv_file))
63     assert len(pts) == 4
64     assert pts[0].visible is False
65     assert pts[1].visible is True and pts[1].x == 100 and pts[1].y == 200
66     assert pts[3].visible is False
67
68
69 def test_trajectory_to_action_windows_detects_moving_shuttle():
70     # 30 fps, 1280px wide. Shuttle moves ~60px/frame while visible (frames 0..29) =>
71     # well above the velocity threshold; then disappears. Expect one window ~1s long.
72     fps, width = 30.0, 1280.0
73     pts = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
74 range(30)]
75     pts += [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(30, 60)]
76
77     windows = TrackNetRunner.trajectory_to_action_windows(
78         pts, fps=fps, frame_width=width, velocity_thresh=0.02,
79         merge_gap=2.0, min_window_duration=0.5, chunk_index=0,
80     )
81     assert len(windows) == 1
82     w = windows[0]
83     assert w["start"] < w["end"]
84     assert w["active_frame_count"] > 0
85     assert w["peak_velocity"] > 0.02
86     assert w["chunk_index"] == 0
87
88 def test_trajectory_to_action_windows_ignores_stationary_shuttle():
89     # Visible but barely moving (1px/frame) => below threshold => no windows.
90     pts = [TrajectoryPoint(frame=i, visible=True, x=500 + i, y=300) for i in range(60)]
91     windows = TrackNetRunner.trajectory_to_action_windows(
92         pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.05,
93         merge_gap=2.0, min_window_duration=0.5,
94     )
95     assert windows == []
96
97
98 def test_trajectory_to_action_windows_splits_on_gap():

```

```

99         # Two bursts of motion separated by a long invisible gap => two windows.
100         burst1 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
range(20)]
101         gap = [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(20, 140)]
102         burst2 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - 140), y=300)
103                     for i in range(140, 160)]
104         windows = TrackNetRunner.trajectory_to_action_windows(
105             burst1 + gap + burst2, fps=30.0, frame_width=1280.0,
106             velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3,
107         )
108         assert len(windows) == 2
109         assert windows[0]["end"] < windows[1]["start"]
110
111
112         # ----- WSL subprocess glue
113
114         def _runner():
115             return TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5"))
116
117
118         def test_healthcheck_ok():
119             r = _runner()
120             with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF 2.17.0 GPUs 1")):
121                 ok, msg = r.healthcheck()
122                 assert ok is True
123                 assert "TF 2.17.0" in msg
124
125
126         def test_healthcheck_failure_returns_message():
127             r = _runner()
128             with patch.object(r, "_wsl_bash", return_value=_proc(1,
stderr="ModuleNotFoundError")):
129                 ok, msg = r.healthcheck()
130                 assert ok is False
131                 assert "ModuleNotFoundError" in msg
132
133
134         def test_healthcheck_no_wsl_binary():
135             r = _runner()
136             with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
137                 ok, msg = r.healthcheck()
138                 assert ok is False
139                 assert "wsl" in msg.lower()
140
141
142         def test_run_predict_requires_weights():
143             r = TrackNetRunner(TrackNetConfig(weights_path="")) # no weights configured
144             assert r.run_predict(r"C:\v\clip.mp4", "out") is None
145
146
147         def test_run_predict_rejects_bad_extension(tmp_path):
148             r = _runner()
149             assert r.run_predict(r"C:\v\clip.mkv", str(tmp_path)) is None
150
151
152         def test_run_predict_success(tmp_path):
153             r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
154             out_dir = str(tmp_path)
155             # predict.py would write "<stem>_predict.csv"; create it so the existence check
passes.
156             csv_path = os.path.join(out_dir, "clip_predict.csv")
157             with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="Prediction
complete.))):
158                 open(csv_path, "w").close()
159                 result = r.run_predict(r"C:\v\clip.mp4", out_dir)

```

```

160         assert result == csv_path
161
162
163     def test_run_predict_failure_returns_none(tmp_path):
164         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
165         with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="CUDA error")):
166             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
167
168
169     def test_run_predict_missing_output_csv(tmp_path):
170         """Exit 0 but no CSV produced should be treated as failure."""
171         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
172         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
173             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
174
175
176     def test_stage_video_success():
177         r = _runner()
178         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="OK")):
179             staged = r._stage_video(r"C:\v\clip.mp4", "clip.mp4")
180             assert staged == "~/clips/clip.mp4"
181
182
183     def test_stage_video_failure():
184         r = _runner()
185         with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="No such file")):
186             assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None
187
188
189     def test_run_predict_stages_when_enabled(tmp_path):
190         """With stage_in_wsl=True, run_predict should stage the video first."""
191         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=True))
192         out_dir = str(tmp_path)
193         csv_path = os.path.join(out_dir, "clip_predict.csv")
194         with patch.object(r, "_stage_video", return_value="~/clips/clip.mp4") as mock_stage,
195             \
196             patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
197             open(csv_path, "w").close()
198             result = r.run_predict(r"C:\v\clip.mp4", out_dir)
199             mock_stage.assert_called_once()
200             assert result == csv_path

```

36. user (2026-06-07T09:11:04.087Z)

```

1     """Tests for the validation registry, focused on the backward-compatible
2     `run_all_with_context` added for observability reporting."""
3     from backend.pipeline.validators.base import ValidationRegistry, ValidationResult,
VideoValidator
4
5
6     class _ProducerValidator(VideoValidator):
7         @property
8         def name(self):
9             return "producer"
10
11         @property
12         def description(self):
13             return "writes ffprobe_meta into the shared context"
14
15         def validate(self, video_path, config, context):
16             context["ffprobe_meta"] = {"fps": 59.94, "width": 1920, "height": 1080}
17             return ValidationResult(validator_name=self.name, passed=True, message="ok")
18
19

```

```

20     class _ConsumerValidator(VideoValidator):
21         @property
22         def name(self):
23             return "consumer"
24
25         @property
26         def description(self):
27             return "reads ffprobe_meta the producer cached"
28
29         def validate(self, video_path, config, context):
30             seen = "ffprobe_meta" in context
31             return ValidationResult(validator_name=self.name, passed=seen,
message=f"saw_meta={seen}")
32
33
34     def _registry():
35         r = ValidationRegistry()
36         r.register(_ProducerValidator())
37         r.register(_ConsumerValidator())
38         return r
39
40
41     def test_run_all_with_context_returns_results_and_context():
42         results, context = _registry().run_all_with_context("v.mp4", {})
43         assert [r.validator_name for r in results] == ["producer", "consumer"]
44         assert context["ffprobe_meta"]["fps"] == 59.94
45         assert results[1].passed is True # consumer saw the shared context
46
47
48     def test_run_all_is_unchanged_and_returns_only_results():
49         results = _registry().run_all("v.mp4", {})
50         assert isinstance(results, list)
51         assert all(isinstance(r, ValidationResult) for r in results)
52         assert [r.validator_name for r in results] == ["producer", "consumer"]
53
54
55     def test_validator_exception_is_captured_not_raised():
56         class _Boom(VideoValidator):
57             @property
58             def name(self):
59                 return "boom"
60
61             @property
62             def description(self):
63                 return "raises"
64
65             def validate(self, video_path, config, context):
66                 raise RuntimeError("explode")
67
68         r = ValidationRegistry()
69         r.register(_Boom())
70         results, _ = r.run_all_with_context("v.mp4", {})
71         assert results[0].passed is False
72         assert "explode" in results[0].message
73

```

37. user (2026-06-07T09:11:04.561Z)

```

1     """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3     Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4     detection accumulation on replay, crash-truncation recovery, manifest
5     compatibility/invalidation, and atomic trajectory writes.
6     """
7     import json

```

```

8     import os
9     import os.path as osp
10
11     import numpy as np
12
13     from backend.pipeline.detectors import wasb_infer as wi
14
15
16     def _win(idx, frames):
17         """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
18         return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
19
20
21     def _write_windows(cache_dir, windows):
22         det_jsonl, _ = wi._cache_paths(cache_dir)
23         with open(det_jsonl, "a") as f:
24             wi._append_windows(f, windows)
25
26
27     # ----- #
28     # detection (de)serialization
29     # ----- #
30     def test_det_plain_and_back_roundtrip():
31         det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
32         plain = wi._det_plain(det)
33         assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
34
35         back = wi._dets_to_tracker([plain])
36         assert len(back) == 1
37         assert isinstance(back[0]["xy"], np.ndarray) # tracker does vector math on xy
38         assert back[0]["xy"].tolist() == [439.0, 64.5]
39         assert back[0]["scale"] == 0
40
41
42     # ----- #
43     # cache replay + resume index
44     # ----- #
45     def test_replay_accumulates_detections_across_windows(tmp_path):
46         cache = str(tmp_path / "c")
47         os.makedirs(cache)
48         # frame 1 appears in BOTH windows -> its candidates must accumulate in window order;
49         # frame 2 is empty in window 0 but detected in window 1.
50         _write_windows(cache, [
51             _win(0, [(0, [[10, 20, 0.9, 0]]), (1, [[11, 21, 0.8, 0]]), (2, [])]),
52             _win(1, [(1, [[12, 22, 0.7, 0]]), (2, [[13, 23, 0.6, 0]]), (3, [])]),
53         ])
54         windows_done, det = wi.load_and_clean_cache(cache)
55         assert windows_done == 2
56         assert det[0] == [[10, 20, 0.9, 0]]
57         assert det[1] == [[11, 21, 0.8, 0], [12, 22, 0.7, 0]] # window-0 then window-1
58         assert det[2] == [[13, 23, 0.6, 0]]
59         assert det[3] == [] # seen but never detected
60
61
62     def test_resume_index_equals_window_count(tmp_path):
63         cache = str(tmp_path / "c")
64         os.makedirs(cache)
65         _write_windows(cache, [_win(i, [(i, [[i, i, 0.5, 0]])]) for i in range(5)])
66         windows_done, _ = wi.load_and_clean_cache(cache)
67         assert windows_done == 5
68
69
70     def test_empty_cache(tmp_path):
71         cache = str(tmp_path / "c")
72         os.makedirs(cache)

```

```

73     windows_done, det = wi.load_and_clean_cache(cache)
74     assert windows_done == 0 and dict(det) == {}
75
76
77     # ----- #
78     # crash robustness
79     # ----- #
80     def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
81         cache = str(tmp_path / "c")
82         os.makedirs(cache)
83         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
84                                _win(1, [(1, [[2, 2, 0.8, 0]])] )])
85         det_jsonl, _ = wi._cache_paths(cache)
86         with open(det_jsonl, "a") as f: # simulate a crash mid-flush
87             f.write('{ "w":2, "f": [[2, [[3, 3, '])
88         windows_done, det = wi.load_and_clean_cache(cache)
89         assert windows_done == 2 # only the two clean windows
survive
90         assert det[1] == [[2, 2, 0.8, 0]]
91         # file is rewritten to the clean prefix -> appending window 2 stays contiguous
92         with open(det_jsonl) as f:
93             lines = [json.loads(l) for l in f if l.strip()]
94             assert [o["w"] for o in lines] == [0, 1]
95             _write_windows(cache, [_win(2, [(2, [[3, 3, 0.7, 0]])] )])
96             assert wi.load_and_clean_cache(cache)[0] == 3
97
98
99     def test_noncontiguous_window_index_stops_replay(tmp_path):
100         cache = str(tmp_path / "c")
101         os.makedirs(cache)
102         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
103                                _win(5, [(5, [[2, 2, 0.8, 0]])] )]) # gap -> stop after w=0
104         windows_done, det = wi.load_and_clean_cache(cache)
105         assert windows_done == 1
106         assert 5 not in det
107
108
109     # ----- #
110     # manifest: atomic write + compatibility / invalidation
111     # ----- #
112     def test_manifest_roundtrip_and_compat(tmp_path):
113         cache = str(tmp_path / "c")
114         os.makedirs(cache)
115         m = {
116             "version": wi.CACHE_VERSION,
117             "frames_dir": osp.abspath("/x/frames"),
118             "weights": "w.pth.tar",
119             "sport": "badminton",
120             "frames_in": 3,
121             "frames_total": 100,
122             "windows_total": 98,
123             "windows_done": 42,
124         }
125         wi.write_manifest(cache, m)
126         assert not osp.exists(osp.join(cache, wi._MANIFEST_FILE + ".tmp")) # atomic, no
tmp left
127         assert wi.read_manifest(cache) == m
128
129         base = dict(frames_dir="/x/frames", weights="w.pth.tar", sport="badminton",
130                     frames_in=3, frames_total=100)
131         assert wi.manifest_compatible(m, **base)
132         assert not wi.manifest_compatible(m, **{**base, "frames_total": 101}) # more
frames
133         assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})
134         assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})

```

```

135         assert not wi.manifest_compatible({}, **base)                                # no cache
136         assert not wi.manifest_compatible(**m, "version": 999}, **base)              # schema bump
137
138
139     def test_read_manifest_missing_returns_none(tmp_path):
140         assert wi.read_manifest(str(tmp_path)) is None
141
142
143     def test_reset_cache_removes_files(tmp_path):
144         cache = str(tmp_path / "c")
145         os.makedirs(cache)
146         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]]]])])
147         wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
148         wi.reset_cache(cache)
149         det_jsonl, mpath = wi._cache_paths(cache)
150         assert not osp.exists(det_jsonl) and not osp.exists(mpath)
151
152
153     # ----- #
154     # misc helpers
155     # ----- #
156
157     def test_default_cache_dir_is_next_to_output():
158         out = osp.join("some", "dir", "sample_long_traj.csv")
159         cd = wi.default_cache_dir(out)
160         assert osp.basename(cd) == "sample_long_traj_wasbcache"
161         assert osp.dirname(cd) == osp.dirname(osp.abspath(out))
162
163
164     def test_atomic_write_trajectory(tmp_path):
165         out = str(tmp_path / "out" / "traj.csv")
166         wi._atomic_write_trajectory(out, [(0, 1, 439.0, 64.0), (1, 0, -1, -1)])
167         assert not osp.exists(out + ".tmp")
168         with open(out) as f:
169             text = f.read().splitlines()
170             assert text[0] == "Frame,Visibility,X,Y"
171             assert text[1] == "0,1,439.0,64.0"
172             assert text[2] == "1,0,-1,-1"

```

38. user (2026-06-07T09:11:05.036Z)

```

1     import pytest
2     from unittest.mock import MagicMock, patch
3     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
4
5
6     def _make_cap_mock(num_frames=300):
7         """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
8         import cv2
9         cap_mock = MagicMock()
10        cap_mock.isOpened.return_value = True
11
12        def mock_get_prop(prop):
13            if prop == cv2.CAP_PROP_FPS:
14                return 30.0
15            if prop == cv2.CAP_PROP_FRAME_WIDTH:
16                return 1000.0
17            return 0.0
18
19        cap_mock.get.side_effect = mock_get_prop
20        cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
21        return cap_mock
22
23
24    def _make_detections(num_frames=300, step=5, track_id=1):

```



```

25     """Per-frame return values for a mocked _detect_and_track: one steadily-moving
26     person. Each entry is the list of (track_id, bbox) for that frame, where the box
27     advances `step` px/frame so its normalised velocity clears the test thresholds.
28     """
29     out = []
30     for i in range(num_frames):
31         x = i * step
32         out.append([(track_id, (float(x), 0.0, float(x + 10), 10.0))])
33     return out
34
35
36 def _mock_stage1(segmenter, num_frames=300, step=5):
37     """Wire a segmenter's Stage-1 detection+tracking to deterministic fakes so tests
38     never need torch model inference, GPU, or a real supervision tracker."""
39     segmenter.model = MagicMock() # short-circuits _lazy_load_model
40     segmenter._make_tracker = MagicMock(return_value=MagicMock())
41     segmenter._detect_and_track = MagicMock(side_effect=_make_detections(num_frames,
step))
42
43
44 @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
45 @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
46 @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
47 @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
48 @patch("os.environ.get")
49 def test_yolo_hybrid_segmenter(mock_env_get, mock_provider_registry, mock_extract,
50                               mock_get_dur, mock_cap):
51     mock_env_get.return_value = "dummy_api_key"
52     mock_get_dur.return_value = 10.0
53     mock_extract.return_value = True
54
55     # Mock provider returned play segment
56     mock_provider = MagicMock()
57     mock_provider.analyze_video.return_value = ([
58         {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
59     ], {})
60     mock_provider_registry.get.return_value = mock_provider
61
62     db_mock = MagicMock()
63     db_mock.add_play_segment.return_value = "yolo_segment_1"
64
65     config = {
66         "indexing": {
67             "use_gpu": False,
68             "chunk_duration": 100,
69             "chunk_overlap": 10,
70             "yolo_velocity_threshold": 0.1,
71             "dead_time_merge_gap": 2.0,
72             # Process every frame so all 300 frames are tracked
73             "yolo_frame_skip": 1,
74             "min_action_window_duration": 1.0,
75         }
76     }
77
78     segmenter = HybridYoloSegmenter(db_mock, config)
79     _mock_stage1(segmenter)
80     mock_cap.return_value = _make_cap_mock()
81
82     res = segmenter.process_video("v1", "dummy.mp4")
83
84     assert len(res) == 1
85     assert res[0]["id"] == "yolo_segment_1"
86     assert res[0]["start_time"] == 0.0
87     assert res[0]["end_time"] == 3.0
88

```

```

89
90 @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
91 @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
92 @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
93 @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
94 @patch("os.environ.get")
95 def test_yolo_hybrid_no_api_key(mock_env_get, mock_provider_registry, mock_extract,
96                               mock_get_dur, mock_cap):
97     """Segmenter should return empty when GEMINI_API_KEY is missing."""
98     mock_env_get.return_value = None
99
100     db_mock = MagicMock()
101     config = {"indexing": {"use_gpu": False}}
102
103     segmenter = HybridYoloSegmenter(db_mock, config)
104     segmenter.model = MagicMock()
105
106     res = segmenter.process_video("v1", "dummy.mp4")
107     assert res == []
108
109
110 @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
111 @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
112 @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
113 @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
114 @patch("os.environ.get")
115 def test_yolo_hybrid_drops_bad_timestamps(mock_env_get, mock_provider_registry,
116 mock_extract,
117                                           mock_get_dur, mock_cap):
118     """Play segments with unparseable timestamps should be dropped, not silently
119 kept."""
120     mock_env_get.return_value = "dummy_api_key"
121     mock_get_dur.return_value = 10.0
122     mock_extract.return_value = True
123
124     # Provider returns one good segment and one with garbage timestamps
125     mock_provider = MagicMock()
126     mock_provider.analyze_video.return_value = ([
127         {"start_time": 1.0, "end_time": 3.0},
128         {"start_time": "N/A", "end_time": "??"},
129     ], {})
130     mock_provider_registry.get.return_value = mock_provider
131
132     db_mock = MagicMock()
133     db_mock.add_play_segment.return_value = "segment_ok"
134
135     config = {
136         "indexing": {
137             "use_gpu": False,
138             "chunk_duration": 100,
139             "chunk_overlap": 10,
140             "yolo_velocity_threshold": 0.05,
141             "dead_time_merge_gap": 2.0,
142             "yolo_frame_skip": 1,
143             "min_action_window_duration": 0.5,
144         }
145     }
146
147     segmenter = HybridYoloSegmenter(db_mock, config)
148     _mock_stage1(segmenter)
149     mock_cap.return_value = _make_cap_mock()
150
151     res = segmenter.process_video("v1", "dummy.mp4")
152
153     # Only the valid segment should survive ? the "N/A" one should be dropped

```

```

152     assert len(res) == 1
153     assert res[0]["start_time"] == 1.0
154
155
156     @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
157     def test_extract_action_windows_returns_enriched_dicts(mock_cap):
158         """_extract_action_windows_from_chunk should return dicts with motion stats."""
159         config = {"indexing": {"min_action_window_duration": 1.0}}
160         segmenter = HybridYoloSegmenter(MagicMock(), config)
161         _mock_stage1(segmenter)
162         mock_cap.return_value = _make_cap_mock()
163
164         windows = segmenter._extract_action_windows_from_chunk(
165             "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
166             frame_skip=1, chunk_index=2,
167         )
168
169         assert len(windows) >= 1
170         w = windows[0]
171         assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
172             "mean_velocity", "track_id_count", "chunk_index"}
173         assert w["chunk_index"] == 2
174         assert w["start"] >= 5.0 # offset into full-video coordinates
175         assert w["active_frame_count"] > 0
176         assert w["peak_velocity"] >= w["mean_velocity"] > 0
177         assert w["track_id_count"] == 1
178
179
180     @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
181     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
182     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
183     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
184     @patch("os.environ.get")
185     def test_yolo_hybrid_persists_and_links_candidates(mock_env_get, mock_provider_registry,
186         mock_extract, mock_get_dur,
187         mock_cap):
188         """Raw candidates should be stored, and confirmed ones linked to their segment."""
189         mock_env_get.return_value = "dummy_api_key"
190         mock_get_dur.return_value = 10.0
191         mock_extract.return_value = True
192
193         mock_provider = MagicMock()
194         mock_provider.analyze_video.return_value = ([
195             {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
196         ], {})
197         mock_provider_registry.get.return_value = mock_provider
198
199         db_mock = MagicMock()
200         db_mock.add_candidate_segment.return_value = 77
201         db_mock.add_play_segment.return_value = 101
202
203         config = {
204             "indexing": {
205                 "use_gpu": False,
206                 "chunk_duration": 100,
207                 "chunk_overlap": 10,
208                 "yolo_velocity_threshold": 0.05,
209                 "dead_time_merge_gap": 2.0,
210                 "yolo_frame_skip": 1,
211                 "min_action_window_duration": 1.0,
212             }
213         }
214
215         segmenter = HybridYoloSegmenter(db_mock, config)
216         _mock_stage1(segmenter)

```

```

216     mock_cap.return_value = _make_cap_mock()
217
218     res = segmenter.process_video("v1", "dummy.mp4")
219
220     assert len(res) == 1
221     # Candidate persisted with the right detector + reproducibility params
222     assert db_mock.add_candidate_segment.called
223     _, kwargs = db_mock.add_candidate_segment.call_args
224     assert kwargs["detector"] == "yolo_hybrid"
225     assert "peak_velocity" in kwargs["stats"]
226     assert kwargs["detection_params"]["frame_skip"] == 1
227     # Confirmed segment linked back to its candidate
228     db_mock.link_candidate_to_segment.assert_called_once_with(77, 101)
229
230
231     @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
232     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
233     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
234     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
235     @patch("os.environ.get")
236     def test_yolo_hybrid_store_candidates_disabled(mock_env_get, mock_provider_registry,
237                                                  mock_extract, mock_get_dur, mock_cap):
238         """store_yolo_candidates=False should skip DB persistence (and linking)."""
239         mock_env_get.return_value = "dummy_api_key"
240         mock_get_dur.return_value = 10.0
241         mock_extract.return_value = True
242
243         mock_provider = MagicMock()
244         mock_provider.analyze_video.return_value = ([
245             {"start_time": 0.0, "end_time": 3.0}
246         ], {})
247         mock_provider_registry.get.return_value = mock_provider
248
249         db_mock = MagicMock()
250         db_mock.add_play_segment.return_value = 101
251
252         config = {
253             "indexing": {
254                 "use_gpu": False,
255                 "chunk_duration": 100,
256                 "chunk_overlap": 10,
257                 "yolo_velocity_threshold": 0.05,
258                 "dead_time_merge_gap": 2.0,
259                 "yolo_frame_skip": 1,
260                 "min_action_window_duration": 1.0,
261                 "store_yolo_candidates": False,
262             }
263         }
264
265         segmenter = HybridYoloSegmenter(db_mock, config)
266         _mock_stage1(segmenter)
267         mock_cap.return_value = _make_cap_mock()
268
269         segmenter.process_video("v1", "dummy.mp4")
270
271         db_mock.add_candidate_segment.assert_not_called()
272         db_mock.link_candidate_to_segment.assert_not_called()
273
274
275     def test_detect_and_track_filters_persons_and_returns_track_ids():
276         """_detect_and_track should keep only confident PERSON detections, feed them to
277         the tracker, and return (track_id, bbox) tuples ? exercising the real torchvision
278         output-shape handling and the COCO person==1 filter (the YOLO-was-0 gotcha)."""
279         import numpy as np
280         from backend.pipeline.segmenters.yolo_hybrid import _PERSON_CLASS_ID

```

```

281
282     segmenter = HybridYoloSegmenter(MagicMock(), {"indexing": {}})
283     segmenter.device = "cpu"
284
285     # Fake torchvision detector output: a person (kept), a low-score person (dropped),
286     # and a racket/other class (dropped).
287     out = {
288         "boxes": MagicMock(),
289         "labels": MagicMock(),
290         "scores": MagicMock(),
291     }
292     out["boxes"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
293         [[0, 0, 10, 10], [5, 5, 15, 15], [20, 20, 30, 30]], dtype=float)
294     out["labels"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
295         [_PERSON_CLASS_ID, _PERSON_CLASS_ID, 38]) # 38 = sports-ball-ish, not person
296     out["scores"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
297         [0.9, 0.1, 0.99])
298     segmenter.model = MagicMock(return_value=out)
299
300     # Fake tracker echoes the detections it receives, assigning a single track id.
301     tracker = MagicMock()
302     tracked = MagicMock()
303     tracked.__len__.return_value = 1
304     tracked.tracker_id = [7]
305     tracked.xyxy = np.array([[0, 0, 10, 10]], dtype=float)
306     tracker.update_with_detections.return_value = tracked
307
308     with patch("backend.pipeline.segmenters.yolo_hybrid.cv2.cvtColor",
309 return_value=np.zeros((10, 10, 3))), \
310         patch("backend.pipeline.segmenters.yolo_hybrid.torch") as mock_torch, \
311         patch.dict("sys.modules", {"supervision": MagicMock()}):
312         mock_torch.no_grad.return_value.__enter__ = lambda s: None
313         mock_torch.no_grad.return_value.__exit__ = lambda s, *a: None
314         # torch.from_numpy(...).permute(...).float().div(...).to(...) -> dummy tensor
315         mock_torch.from_numpy.return_value.permute.return_value.float.return_value.div.r
316         eturn_value.to.return_value = "tensor"
317         result = segmenter._detect_and_track("frame", tracker, score_thresh=0.5)
318
319     # Only the high-confidence person survived the score+class filter
320     assert result == [(7, (0.0, 0.0, 10.0, 10.0))]
321     # And exactly that filtered set (1 box) was handed to the tracker
322     tracker.update_with_detections.assert_called_once()
323

```

39. user (2026-06-07T09:11:05.604Z)

```

1     """Golden guard: every footgun rule's faq_ref must resolve to a real README heading.
2
3     This is the committed backbone of report debuggability ? if a rule cites README#Qn,
4     that section must exist, so a reader who follows the pointer always lands somewhere.
5     (A fresh-reader eval found a rule citing the wrong-but-existing section; an existence
6     test can't catch *semantic* mismatch, but it locks the floor: no dangling anchors.)
7     """
8     import re
9     from pathlib import Path
10
11     import pytest
12
13     pytest.importorskip("obsreport")
14     from obsreport import load_spec # noqa: E402
15
16     ROOT = Path(__file__).resolve().parent.parent
17     SPEC = ROOT / "backend" / "reporting" / "spec.yaml"
18     README = ROOT / "README.md"
19

```

```

20
21     def _headings(md: str):
22         return [ln.lstrip("#").strip() for ln in md.splitlines() if
ln.lstrip().startswith("#")]
23
24
25     def _slug(h: str) -> str:
26         s = re.sub(r"^\w\s-", "", h.lower())
27         return re.sub(r"\s+", "-", s.strip())
28
29
30     def _anchor_resolves(anchor: str, headings) -> bool:
31         a = anchor.lower()
32         for h in headings:
33             hl = h.lower()
34             if hl == a or hl.startswith(a + ":") or hl.startswith(a + " ") or
hl.startswith(a + "."):
35                 return True # Q-number style: "Q7" -> "### Q7: ..."
36             if _slug(h) == a:
37                 return True # full GitHub slug match
38         return False
39
40
41     def test_every_rule_faq_ref_resolves_to_a_readme_heading():
42         spec = load_spec(SPEC)
43         headings = _headings(README.read_text(encoding="utf-8"))
44         unresolved = []
45         for rule in spec.rules:
46             if not rule.faq_ref:
47                 continue
48             file_part, _, anchor = rule.faq_ref.partition("#")
49             assert file_part in ("README", "README.md"), f"{rule.code}: unexpected faq file
'{file_part}'"
50             if not _anchor_resolves(anchor, headings):
51                 unresolved.append((rule.code, rule.faq_ref))
52         assert not unresolved, f"faq_ref anchors with no matching README heading:
{unresolved}"
53
54
55     def test_actionable_rules_have_faq_refs():
56         """error/warn findings must cite a FAQ (info may omit). Belt-and-suspenders on top
57         of obsreport's own spec-load validation."""
58         spec = load_spec(SPEC)
59         missing = [r.code for r in spec.rules if r.severity in ("error", "warn") and not
r.faq_ref]
60         assert not missing, f"actionable rules missing faq_ref: {missing}"
61

```

40. assistant (2026-06-07T09:12:47.907Z)

I now have all the test files read. Here is the dense CODE_MAP chunk for the Tests subsystem.

Tests (test ? subsystem map)

`tests/` ? pytest suite. Pure-logic + mocked-boundary unit tests; no WSL/GPU/Gemini/ffmpeg actually invoked (all I/O boundaries patched). Each file maps to one production subsystem. \$marks data contracts the tests pin; ? marks registry/plugin extension points exercised; ? marks footguns the tests deliberately lock down.

- `@tests/\_\_init\_\_.py` ? empty package marker (`# Tests package`).

Eval / metrics subsystem (`backend/eval/\*`)

- `@tests/test_eval_metrics.py` ? covers `backend/eval/metrics` (pure interval matching). Tests `interval_iou`, `match_intervals` (? `MatchResult.matches`/`.unmatched_pred_indices`/`.unmatched_gt_indices`, `.matches`[].`pred_index`), `score` (? `Score.precision/recall/f1/mean_iou/mean_start_error/mean_end_error/true_positives/false_positives/false_negatives`, `.as_dict()`), `pr_curve`. `$score.as_dict()` keys incl. `iou_threshold`. ? greedy matcher prefers higher-IoU pred; one GT not double-claimed; default `pr_curve` = 9 thresholds; degenerate/zero-width intervals ? IoU 0.
- `@tests/test_eval_batch.py` ? covers `backend/eval/batch`. `batch.resolve_video_path` (? normalizes `./data\..` Windows-style relative paths against a collector root; absolute kept; `None/""` ? `None`), `batch.default_stages_for` (? `auto` ? `["final"]` for `motion`/`gemin`, `["candidates","final"]` for two-stage `yolo_hybrid`; explicit overrides auto), `batch.aggregate` (\$per-video shape `{"stages":{"final":{"score":{"tp,fp,fn,mean_iou,f1}}}}`; emits `micro_precision/micro_recall/micro_f1`, TP-weighted `mean_iou`, `macro_f1`, `videos`; handles empty list / zero-division).
- `@tests/test_eval_classifier.py` ? covers `backend/eval/classifier::LogisticRegression` (numpy-only impl). `.fit(X,y,feature_names=,class_weight=)`, `.predict`, `.predict_proba` (range [0,1]), `.save`/`.load` (\$JSON roundtrip preserves `feature_names`), ? `predict_proba` before fit raises `RuntimeError`; `fit` on empty raises `ValueError`; `class_weight="balanced"` recovers 10:1 minority.
- `@tests/test_eval_training.py` ? covers `backend/eval/training`. `extract_yolo_features` (\$row shape `{start_time,end_time,stats:{peak_velocity,mean_velocity,active_frame_count,track_id_count}}` ? 5-vec; missing stats default 0), `YOLO_FEATURE_NAMES`, `label_candidates` (IoU-threshold labeling, one-to-one best match), `build_training_set` (? `(X,y,intervals)`; `feature_fn=` override). ? feature order is `[dur,peak,mean,frames,tracks]`.
- `@tests/test_eval_annotations.py` ? covers `backend/eval/annotations`. `parse_timestamp` (decimal/`MM:SS`/`HH:MM:SS`; ? raises on `"/"abc"/4-part`), `load_annotations` (\$CSV `start,end`; header auto-detected, blank/`#`comment lines skipped, sorted by start; ? rejects `start>end` and short rows), `write_scaffold` (header-only template loads to `[]`).
- `@tests/test_eval_metrics.py` + `@tests/test_eval_harness.py` are the keystone scorer; harness file ? covers `backend/eval/harness`: `evaluate` (? `Report.stages[stage].score`/`.pr_curve`), `get_predictions` (stages `final`/`candidates` w/ `detector=` filter; ? unknown stage raises `ValueError`), `format_report_text` (`show_failures=` lists "missed rallies" w/ `HH:MM:SS`), `report_to_dict` (\$JSON-serialisable, `video_id`/`stages.<s>.score.f1`), `_rows_to_intervals` (sorts). Also exercises CLI `@run_eval.py::main` (? exit codes: 0 ok, 2 missing-annotations / missing-db; `--scaffold`/`--json` flags) and `run_eval.get_file_md5` (32-char stable md5).
- `@tests/test_eval_regression.py` ? covers `backend/eval/regression` (GS-2 regression runner) + CLI `@run_regression.py::main`. `save_baseline`/`load_baselines` (\$baseline store keyed `"<vid>::<stage>"` ? `{precision,recall,f1}`; absent ? `{}`), `regress` (? `RegressionResult.regressed/.precision/.recall/.reasons`; ? "no baseline" is NOT a regression; recall drop beyond `tolerance` regresses w/ reason mentioning "recall"; default tol 0.05), `regress_with_provider` (tier="file" via GS-1 FileProvider). ? CLI exit-code CI gate: `--update-baseline` ? 0; regression ? 1, prints "REGRESSED".
- `@tests/test_calibration.py` ? covers `backend/eval/calibration` (overfitting guard). `pct_improvement` (? from-zero ? `inf`; 0?0 ? 0.0), `kfold_indices` (? caps k to n; k=0 ? []; train/test disjoint & cover all), `select_best` (picks perfect config by `metric=`), `cross_validate` (\$result `{metric,folds,mean_test,generalization_gap,picked_configs}`; default `metric="recall"`; too-few ? `{"error":...}`), `leave_one_video_out` (\$video dict `{name,predict_fn,gts}`; ? `{mean_held_out,per_video[].config}`; needs ?2), `improvement_report` (? `{baseline,tuned,precision_delta_pts,precision_delta_pct}`).
- `@tests/test_calibrate_wasb.py` ? covers `backend/eval/calibrate_wasb` (WASB calibration driver; no WSL/GPU). `load_golden_gts` (\$golden CSV `rally_number,start_time,end_time,shots_count`; ? incomplete rows skipped), `make_predict_fn` (trajectory CSV \$Frame,Visibility,X,Y ? moving-segment windows), `run` (? `{best_cfg,baseline:{f1},cv:{mean_test}}`), `BASELINE` const.
- `@tests/test_calibrate_local.py` ? covers `backend/eval/calibrate_local` (local-only, no Gemini). `local_grid` (? size = 5x3x3x4 = velocityxmergexmin_durxmin_crossings), `make_local_predict_fn` (cfg tuple `(velocity,merge,min_dur,min_crossings)`; ? min_crossings gate drops monotonic motion).
- `@tests/test_distill_local.py` ? covers `backend/eval/distill_local` (distilled local classifier). `build_candidates` (? `(intervals,X)`; \$FEATURE_NAMES = 5 invariant features, duration>0), `predict_rallies` (`threshold=` cutoff; ? 1.01 ? []; 0.0 ? merged windows). Imports `backend/eval/classifier::LogisticRegression`.
- `@tests/test_export_wasb_segments.py` ? covers `backend/eval/export_wasb_segments` (eyeball-verification export). `seconds_to_mmss` (? negative clamps to "0:00.000"), `segments_to_rows`

(\$row `{rally\_number,start\_time,end\_time,duration,start\_mmss}`, time-ordered numbering),
`write\_segments\_csv`, `build\_comparison` (? `{score,rows[type?{golden,pred\_extra}],num\_gt}`;
statuses `matched`/`MISS`/`false\_positive`), `run` (`min\_crossings` gate, `baseline=`). ?
keystone reuse claim: exported CSV loads straight into
`backend/tools/rally\_slicer::load\_rally\_labels`.
- `@tests/test\_gemini\_refine.py` ? covers `backend/eval/gemini\_refine` (pure helpers, no API).
`merge\_overlaps` (`gap\_tol=` stitches small gaps, preserves real ones), `\_offset\_segments`
(\$input dicts `{start\_time,end\_time}` ? clip-offset tuples; ? end?start & unparseable dropped;
clamps to clip bounds).

Audio subsystem (`backend/pipeline/audio/\*`, `backend/eval/audio/\*`)

- `@tests/test\_hit\_detector.py` ? covers `backend/pipeline/audio/hit\_detector` (pure DSP).
`detect\_hits` (? `HitEvent.t`; `k=` threshold, `band=` bandpass ? ? broadband clicks survive
band-limit; silence ? []), `onset\_envelope` (? `(env, hop\_sec)`; band changes flux, same
length), `\_moving\_stats`. Also `backend/eval/audio/e1\_probe::cluster\_hits`
(`gap\_sec=`/`min\_hits=`; ? lone hits dropped, cluster spanned+padding). \$`HitEvent(t,
<strength>)`.

Detectors subsystem (`backend/pipeline/detectors/\*`)

- `@tests/test\_tracknet\_runner.py` ? covers `backend/pipeline/detectors/tracknet\_runner` (pure-
Python pieces; WSL subprocess mocked via `\_wsl\_bash`). `\_to\_wsl\_mnt\_path` (? `C:\..` ?
`/mnt/c/..`; POSIX/`~` passthrough), `TrackNetConfig.from\_indexing\_cfg` (\$indexing
`{tracknet:{wsl\_distro,conda\_env,weights\_path,queue\_length,stage\_in\_wsl}}`; defaults
distro=Ubuntu, env=TrackNetV4, weights="" must be user-supplied),
`TrajectoryPoint(frame,visible,x,y)`, `TrackNetRunner.parse\_trajectory\_csv`
(\$`Frame,Visibility,X,Y`; visibility 0?False), `.trajectory\_to\_action\_windows` (\$window
`{start,end,active\_frame\_count,peak\_velocity,chunk\_index}`;
`velocity\_thresh`/`merge\_gap`/`min\_window\_duration`; splits on invisible gap, ignores
stationary), `.healthcheck` (? missing wsl binary ? msg contains "wsl"), `.run\_predict` (?
requires weights, rejects non-`.mp4`, exit-0-but-no-CSV ? None), `.\_stage\_video` (?
`~/clips/<name>`), `stage\_in\_wsl=True` stages first.
- `@tests/test\_rally\_gate.py` ? covers `backend/pipeline/detectors/rally\_gate` (net-crossing
gate; pure). `estimate\_play\_axis` (? `(axis,net,span)`; picks larger-spread axis),
`count\_net\_crossings` (counts sign changes across `net`; ? `deadband` ignores jitter; invisible
points skipped), `gate\_windows` (? `[GateResult.kept/.crossings]`), `apply\_gate` (? filtered
windows; `min\_crossings=`). ? kills single-toss (1 crossing), keeps multi-crossing rally.
Imports `TrajectoryPoint` from tracknet_runner.
- `@tests/test\_wasb\_cache.py` ? covers `backend/pipeline/detectors/wasb\_infer` (resumable cache
layer; pure-Python). `\_det\_plain`/`\_dets\_to\_tracker` (\$det `{xy:ndarray,score,scale}` ? plain
`[x,y,score,scale]`; ? back-conversion restores ndarray for vector math), `load\_and\_clean\_cache`
(? `(windows\_done, det)`; accumulates across windows in order; ? truncated trailing JSONL line
recovered+rewritten to clean prefix; non-contiguous window index stops replay),
`\_cache\_paths`/`\_append\_windows`/`write\_manifest`/`read\_manifest` (? atomic, no `.tmp` left;
missing ? None), `manifest\_compatible` (\$manifest
`{version,frames\_dir,weights,sport,frames\_in,frames\_total,windows\_total,windows\_done}`; ?
invalidated by frames_total/weights/sport mismatch, empty, or `version` bump vs
`CACHE\_VERSION`), `reset\_cache`, `default\_cache\_dir` (? `<stem>wasbcache` next to output),
`\_atomic\_write\_trajectory` (\$writes `Frame,Visibility,X,Y` header).

Segmenters subsystem (`backend/pipeline/segmenters/\*`)

- `@tests/test\_base.py` ? covers `backend/pipeline/segmenters/base`. ?
`SegmenterRegistry.register/get/list\_available` (? name?description map); `BasePlaySegmenter`
ABC (`name`/`.description` props,
`.process\_video(video\_id,video\_path,player\_pool=,max\_frames=)`). Uses local `DummySegmenter`.
- `@tests/test\_gemini.py` ? covers `backend/pipeline/segmenters/gemini::GeminiPlaySegmenter`
(provider_registry + video utils mocked). \$indexing `{chunk\_duration,chunk\_overlap}`. ? large
chunk_duration ? single-file path; small ? chunked w/ per-chunk offset (chunk-2 local 30s ?
global 170s); \$segment metadata `shot\_count` ? provider `shuttle\_exchange\_count`.
- `@tests/test\_yolo\_hybrid.py` ? covers
`backend/pipeline/segmenters/yolo\_hybrid::HybridYoloSegmenter` (Stage-1 detect/track + Gemini
Stage-2; all mocked). `process\_video`, `\_extract\_action\_windows\_from\_chunk` (\$window
`{start,end,active\_frame\_count,peak\_velocity,mean\_velocity,track\_id\_count,chunk\_index}`; offset
into full-video coords), `\_detect\_and\_track` (? COCO `\_PERSON\_CLASS\_ID` filter ? "YOLO-was-0"
gotcha; score threshold; returns `(track\_id,bbox)`). ? no GEMINI_API_KEY ? []; unparseable


```

timestamps dropped; `store_yolo_candidates=False` skips DB persist+link; on success links
candidate?segment (`link_candidate_to_segment(cand_id, seg_id)`), candidate
`detector="yolo_hybrid"`, `detection_params.frame_skip`.
- `@tests/test_tracknet_hybrid.py` ? covers
`backend/pipeline/segmenters/tracknet_hybrid::TrackNetHybridSegmenter` (TrackNetRunner +
provider mocked). ? `_patched` decorator stacks 6 patches ? innermost-first injection order
(gotcha documented inline). `_probe_fps_width` (? fallback `(30.0,1280.0)` when capture fails),
`process_video` (happy path links candidate `detector="tracknet_hybrid" ? segment; no API key ?
[]; healthcheck failure aborts before `run_predict`; no trajectory ? []);
`store_yolo_candidates=False` skips persist; drops unparseable segments), `.name`/`.description`
("TrackNet").

```

Providers subsystem (`backend/providers/\*`)

```

- `@tests/test_providers.py` ? covers `backend/providers`. ?
`provider_registry.list_available/get(name, api_key=, model_name=)` (? instance);
`backend/providers/base::AIVideoProvider` ABC; `gemini` registered ? `GeminiProvider`.
- `@tests/test_provider_gemini.py` ? covers `backend/providers/gemini::GeminiProvider` (`genai`
client mocked). `analyze_video` (? `(segments, metrics{upload_time,process_time,gen_time})`);
`expects` JSON list `[{start_time,end_time}]`. ? non-list JSON ? []; ? rejects >`MAX_UPLOAD_MB`
(500) before upload (`upload.assert_not_called`); upload failure ? []; always
`files.delete(name=...)` cleanup.

```

Annotations subsystem (`backend/annotations/\*`)

```

- `@tests/test_annotation_providers.py` ? covers `backend/annotations` + `file_provider`. ?
`annotation_registry.list_available/get(name, **kwargs)` (? unknown ? `KeyError`);
`AnnotationProvider` ABC; `file_provider::FileProvider` ($submit/poll/fetch contract;
`poll`?`DONE`/`FAILED` from `base`; bare video_id resolved to `

```

Validators subsystem (`backend/pipeline/validators/\*`)

```

- `@tests/test_validators_base.py` ? covers
`backend/pipeline/validators/base::ValidationRegistry`. ?
`register/run_all/run_all_with_context` (? `run_all_with_context` is the obs-reporting addition:
returns `(results, shared_context)` so producer validator can stash `ffprobe_meta` for a
consumer; `run_all` unchanged returns only results).
`$ValidationResult(validator_name,passed,message)`; `VideoValidator` ABC
(`.name`/`.description`/`.validate(video_path,config,context)`). ? validator exception captured
(passed=False, message contains the error), not raised.

```

Config subsystem (`backend/config`)

```

- `@tests/test_config.py` ? covers `backend/config` (Pydantic typed config). `load_config` (?
nonexistent path ? model built-in defaults), `save_default_config`, `AppConfig`. $defaults
pinned: `sport="badminton"`, `indexing.default_segmenter="yolo_hybrid"`,
`validation.min_blur_threshold=20.0`, `indexing.ai_max_workers=5`,
`indexing.tracknet.velocity_threshold=0.02`, `ai_provider="gemini"`,
`validation.min_resolution_width=1280`, `validation.relevance.court_pixels_min_ratio=0.05`,
`stitching.begin_padding=0.5`. ? extra/future keys tolerated; ? `AppConfig.get()` dict-compat
must return nested sections as plain dicts (legacy chained `.get("validation",{ }).get(...)`) via
dynamic section scan, not hardcoded list.

```

Infrastructure subsystem (`backend/infrastructure/database`)

```

- `@tests/test_database.py` ? covers `backend/infrastructure/database::Database`. `__init__`
(creates `videos`/`play_segments`; ? idempotent + upgrades pre-v1 DB to `candidate_segments`;

```

```
`PRAGMA user_version == 1`), `add_video`/`get_video`/`clear_video_data`,
`add_play_segment`/`query_play_segments`/`add_event`, candidate API
`add_candidate_segment`/`query_candidate_segments(detector=,
only_unlabeled=)`/`link_candidate_to_segment`. $`EXPECTED_CANDIDATE_COLUMNS` schema-contract
guard (? any column change must update this set ? id PK, NOT NULL on
video_id/detector/start_time/end_time/duration). ? candidate cascade-deletes on video delete;
`only_unlabeled` excludes rows once `human_label` set; `_get_connection`.
```

Sports profile subsystem (`backend/sports/\*`)

```
- `@tests/test_sport_profiles.py` ? covers `backend/sports`. ?
`sport_registry.list_available/get("badminton")` ? `BadmintonSportProfile`; `base::SportProfile`
ABC. `.name`, `.get_prompt(chunk_duration=)` ($must contain "badminton"/"decimal seconds"/the
chunk value), `.get_relevance_config()` ($keys
`court_green_lower/court_green_upper/court_pixels_min_ratio`).
```

Tools subsystem (`backend/tools/\*`)

```
- `@tests/test_rally_slicer.py` ? covers `backend/tools/rally_slicer` (pure logic, no ffmpeg).
`load_rally_labels` ($CSV `rally_number,start_time,end_time,start_mmss,end_mmss`; decimal or
`m:ss` fallback; ? MISSED/no-end rows & zero-duration skipped; fractional IDs like "16.5"
preserved), `compute_clip_windows` (? `[ClipWindow(rally,start,end)]`);
`BEGIN_PADDING`/`END_PADDING` consts; ? clamps to `video_duration`; `"all"` orders by start-time
not dict order; unknown IDs dropped; window entirely past end skipped).
```

Reporting / observability subsystem (`backend/reporting/\*`, external `obsreport` lib)

```
- `@tests/test_reporting_harvest.py` ? covers `backend/reporting` (`emit_indexer_report`,
`harvest.record_run`). ? `pytest.importorskip("obsreport")` ? soft dep, skips if absent.
$`harvest.record_run(...)` ? `RunRecord.envelope` (fields:
`verdict`{pass,pass_with_warnings,fail}`, `.flags[].code/.faq_ref`,
`.video_meta.fps/.fps_source`{probed,fallback}`,
`.get_stage(name).status/.metrics[].key/.value`, `.lint()`). ? footgun rules under test:
`silent_provider_noop` (no GEMINI_API_KEY + 0 segments, faq_ref `README#Q7`),
`ai_empty_vs_no_rallies` (key present, 0 segments ? warn), `synthetic_motion_metadata` (motion
segmenter; ? provider-noop must NOT fire for non-AI motion), `segmenter_divergence` (fires only
when no explicit override), `reingest_replaced_prior`, `validation_failed` (halts segmentation),
`fps_fallback_used`. `emit_indexer_report` writes `<video>.report.md`/`.summary.md`/json next to
video; ? accepts plain dict OR typed `AppConfig` (normalized); `report.enabled=False` ? returns
None.
- `@tests/test_reporting_spec_faq.py` ? covers `backend/reporting/spec.yaml` ? `README.md`
cross-reference integrity (golden guard). ? `importorskip("obsreport")`; uses
`obsreport.load_spec`. `test_every_rule_faq_ref_resolves_to_a_readme_heading` (? every
`rule.faq_ref` "README#Qn" anchor must resolve to a real README heading ? Q-style prefix or
GitHub slug; locks "no dangling anchors"), `test_actionable_rules_have_faq_refs` (? severity
`error`/`warn` rules must cite a faq_ref; `info` may omit).
$`spec.rules[].code/.faq_ref/.severity`.
```

Utils subsystem (`backend/utils/\*`)

```
- `@tests/test_geometry.py` ? covers `backend/utils/geometry`. `get_center`, `get_distance`,
`get_velocity(bbox1,bbox2,fps)`, `get_velocity_normalised(...,frame_width)` (? width?0 ? falls
back to raw velocity), `calculate_iou` (? uses +1 bound expansion ? comment flags it;
identical?1.0, disjoint?0.0).
- `@tests/test_video.py` ? covers `backend/utils/video` (subprocess mocked).
`get_video_duration` (? ffmpeg exception ? 0.0), `extract_video_chunk(in,start,dur,out)` (?
ffmpeg exception ? False; success ? True, `subprocess.run` called once).
```

Cross-cutting notes / footguns

```
- ? Two stale `.pyc` files in `tests/__pycache__` have no live source: `test_gemini_api.*` and
`test_geometry`/`test_yolo_hybrid` cpython-311 variants ? the current sources are the `.py`
files listed above (no `test_gemini_api.py` exists).
- ? Soft-dep gate: only the two `test_reporting_*` files import `obsreport` (skipped if not
installed); everything else runs with stdlib + numpy/cv2 only.
- ? CLI entrypoints are tested at repo root, not under `tests`:
`@run_eval.py::main`/`get_file_md5` (via test_eval_harness) and `@run_regression.py::main` (via
test_eval_regression) ? both assert exit-code contracts for CI gating.
- Keystone "one-scorer" contract is asserted in three places that all feed
```

```
`backend/eval/metrics::score`: `@tests/test_annotation_providers.py` (provider output shape),  
`@tests/test_export_wasb_segments.py` (export?slicer reuse), `@tests/test_eval_harness.py` (DB  
rows?intervals).
```